

COMSATS University, Islamabad Pakistan

FalconScan

(AI-powered APK Vulnerability Scanner)

By

Mohamed Ahsan **CIIT/FA22-BCT-001/ISB**

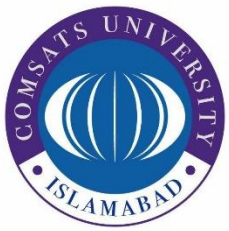
Abdullah **CIIT/FA22-BCT-004/ISB**

Supervisor

Dr. Farhana Jabeen

Bachelor of Science in Cyber Security (2022-2026)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



COMSATS University, Islamabad Pakistan

FalconScan

(AI-powered APK Vulnerability Scanner)

A project presented to

COMSATS University, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Cyber Security (2022-2026)

By

Mohamed Ahsan **CIIT/FA22-BCT-001/ISB**

Abdullah **CIIT/FA22-BCT-004/ISB**

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other, we will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Mohamed Ahsan

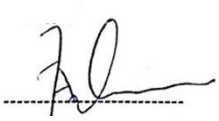


Abdullah



CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CT) FalconScan was developed by **Mohamed Ahsan (CIIT/FA22-BCT-001)** and **Abdullah (CIIT/FA22-BCT-004)** under the supervision of Dr. Farhana Jabeen and that in her opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Cyber Security.



Supervisor

External Examiner

Head of Department

(Department of Computer Science)

EXECUTIVE SUMMARY

With the rapid expansion of Android applications across industries, ensuring mobile app security has become a major challenge. Traditional vulnerability detection tools, such as MobSF and VirusTotal, primarily rely on static rule-based analysis, which is limited to identifying known threats. These approaches struggle to detect advanced or obfuscated vulnerabilities that exhibit malicious behavior only during runtime. Moreover, the manual interpretation of scan results consumes time and requires expert knowledge, making large-scale security testing inefficient.

To overcome these limitations, FalconScan has been developed, an AI-powered APK Vulnerability Scanner designed to automate the detection of both known and unknown security threats in Android applications. FalconScan integrates static and dynamic analysis techniques within a unified hybrid framework, providing comprehensive insights into app security.

The system performs static analysis by decompiling APKs and examining manifest files, permissions, and API usage, while its dynamic analysis engine executes the application in a controlled emulator environment to observe runtime behavior such as network communication, file access, and system calls. Extracted features from both phases are then analyzed using a Large Language Model (LLM), which intelligently classifies vulnerabilities and provides human-readable explanations for its decisions.

FalconScan operates through a web-based interface, allowing users to upload APKs, initiate scans, and view detailed vulnerability reports with severity levels and mitigation recommendations. The platform also maintains a local vulnerability database for quick rule-based detection and supports continuous learning through periodic AI model updates, ensuring adaptability to emerging threats.

By automating complex analysis workflows and incorporating AI reasoning, FalconScan significantly reduces manual effort, increases detection accuracy, and provides actionable insights for developers and cybersecurity professionals. It serves as an innovative, research-oriented, and scalable solution for modern Android application security assessment.

Abstract

FalconScan was developed to provide a unified, practical, and fast security assessment solution for Android application packages amid the growing attack surface created by mobile app dependence. Current APK security practices are fragmented across manual reverse engineering, isolated static scanners, and compliance checklists that demand high expertise while failing to deliver actionable, audience-appropriate findings. Most available tools focus on narrow indicators, produce noisy outputs without risk prioritization, and lack the end-to-end operational support needed for authenticated workflows, scan history, and repeatable assessments. FalconScan targets insecure manifest configurations, over-privileged permissions, weak cryptography, exposed components, hardcoded credentials, and insecure network communication, while also converting raw findings into strategic intelligence through AI-assisted impact explanation, attack vector mapping, and remediation guidance. The core gap it fills is the disconnect between low-level detection outputs and decision-ready, context-rich reporting in environments where both precision and usability are essential. Its first objective is to implement a hybrid analysis pipeline using Django REST APIs, Androguard-powered static inspection, OWASP-oriented vulnerability checks, and weighted risk scoring for reliable threat assessments. Second, it aims to deliver a modern Next.js and TypeScript frontend supporting secure authentication, real-time scan tracking, categorized findings, and professional PDF report export. Third, it seeks to enrich conventional scanning with an AI-driven intelligence layer capable of executive summarization, attack scenario synthesis, compliance commentary, and prioritized remediation roadmaps for both novice and expert users. By consolidating multi-technique detection, risk quantification, and comprehensive reporting into one coherent platform, FalconScan advances mobile software assurance and enables more informed cybersecurity decision-making across development, academic, and organizational contexts.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Dr. Farhana Jabeen”. Without her personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are grateful to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us with the values of honesty & hard work.

Mohamed Ahsan



Abdullah



Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package Kit
AVD	Android Virtual Device
CI	Continuous Integration
CIIT	COMSATS Institute of Information Technology
CPU	Central Processing Unit
CVSS	Common Vulnerability Scoring System
DB	Database
DFA	Dynamic Fuzzing Analysis
DL	Deep Learning
FR	Functional Requirement
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HTTPS	Hypertext Transfer Protocol Secure
ISB	Islamabad Campus
JSON	JavaScript Object Notation
LLM	Large Language Model
MCP	Model Context Protocol
ML	Machine Learning
MobSF	Mobile Security Framework
NLP	Natural Language Processing
NFR	Non-Functional Requirement
OS	Operating System
PDF	Portable Document Format
PER	Performance Requirement
REL	Reliability Requirement

RQ	Requirement
SRS	Software Requirements Specification
SQL	Structured Query Language
TLS	Transport Layer Security
UI	User Interface
UML	Unified Modeling Language
USE	Usability Requirement
VM	Virtual Machine
VMS	Vulnerability Management System
VTS	VirusTotal Scanner
XML	Extensible Markup Language

Table of Contents

Abstract v

1 Chapter 1: Introduction and Problem Definition 8

1.1 Overview of the Project 8
1.2 Vision Statement 9
1.3 Problem Statement 9
1.4 Problem Solution 10
1.5 Objectives of the Proposed System 11
1.6 Scope 11
1.6.1 Limitations/Constraints 12
1.7 Modules 13
1.7.1 Module 1: User Authentication & APK Management 13
1.7.2 Module 2: Static Analysis Engine 13
1.7.3 Module 3: Fuzzing Engine 14
1.7.4 Module 4: Dynamic Analysis Engine 14
1.7.5 Module 5: LLM-Driven Vulnerability Analysis 15
1.7.6 Module 6: Reporting & Dashboard UI 16
1.8 Related System Analysis/Literature Review 16
1.8.1 Literature Review 16
1.8.2 Related System Analysis 18
1.9 Tools and Technologies 19
1.10 Project Contribution 20
1.11 Relevance to Course Modules 21

2 Chapter 2: Requirement Analysis 21

2.1 User Classes and Characteristics 22
2.1.1 Use case Diagram 23
2.1.2 Requirement Identifying Technique 23
2.2 Functional Requirements 24

2.2.1 Mockup M1 – Login Page 25
2.2.2 Mockup M2 – Dashboard Page 26
2.2.3 Mockup M3 – Upload APK Page 27
2.2.4 Mockup M4 – Scan Reports Page 29
2.2.5 Mockup M5 – Scan History Page 31
2.2.6 Event–Response Table for Backend Processes 32
2.3 Non-Functional Requirements 33
2.3.1 Reliability 33
2.3.2 Usability 33
2.3.3 Performance 33
2.3.4 Security 34
2.4 External Interface Requirements 34
2.4.1 User Interface Requirements 34
2.4.2 Software Interfaces 35
2.4.3 Hardware Interfaces 35
2.4.4 Communication Interfaces 35
3 Chapter 3: Design and Architecture 37
3.1 System Architecture Overview 37
3.1.1 Purpose 38
3.2 Guidelines for Students 38
3.2.1 Conceptual Architecture Diagram 39
3.2.2 Technologies and Services 41
3.3Architecture Style / Pattern 42
3.3.1 Selected Architecture Pattern & Justification 42
3.3.2 Architecture Style Diagrams 43
3.4 Design Models 44
3.4.1 Design Models for Object Oriented Development Approach 45
3.5 Data Design 64
3.5.1 Major Entity Storage 64
3.5.2 Data Dictionary 65

4 Chapter 4: Project Methodology & Algorithms..... 73
4.1.1 Project Methodology (Step-by-Step Approach).....73
1. Data Collection 73
2. Data Preprocessing.....74
3. Feature Extraction & Selection.....74
4. Model Training and AI-Based Analysis 75
5. Advanced Techniques.....76
6. Deployment.....76
4.1.2 Algorithm 77
Algorithm 1: Vulnerability Classification (AI-Based) 77
Algorithm 2: Static Analysis & Feature Extraction.....78
Algorithm 3: Dynamic Behavior Monitoring 78
4.2 Training Results and Model Evaluation.....79
4.3 Security Techniques 83
4.3.1 Authentication Techniques 83
4.3.3 Attack Prevention Techniques 84
4.3.4 Integration with AI Model for Security 86
4.4 External APIs and SDKs 86
4.4.1 Axios (HTTP Client for API Requests).....86
4.4.2 JWT (JSON Web Token).....87
4.4.3 Strace and Ltrace (System Call Monitoring) 87
4.4.4 APKTool and JADX (Static Analysis) 88
4.4.5 Django REST Framework (Backend API) 88
4.4.6 Summary of APIs/SDKs.....89
4.5 User Interface 90
4.5.1 Login Screen 90
4.5.2 Home Dashboard 92
4.5.3 Upload APK Screen.....93
4.5.4 Scan Execution Screen.....94
4.5.5 Report Screen.....95
4.5.6 Scan History Screen.....96

5 Chapter 5: Testing and Evaluation 98

5.1 Unit Testing 98

5.2 Functional Testing 101

5.3 Business Rules Testing 102

5.4 Integration Testing 103

6 Chapter 6: Conclusion and Future Work 106

6.1 Conclusion 106

6.2 Future Work 107

7 Chapter 7: References 108

8 Chapter 8: Plagiarism Report 110

List of Figures

Figure 2.1: Use case diagram 23
Figure 2.2: Login page 25
Figure 2.3: Dashboard Page 26
Figure 2.4: Upload APK Page 27
Figure 2.5: Scan Reports Page 29
Figure 2.6: Scan History Page 31
Figure 3.1: Conceptual Architecture diagram 39
Figure 3.2: Pipeline Architecture 43
Figure 3.3: Layered Architecture inside the backend 44
Figure 3.4: Activity Diagram 1 – Apk upload 45
Figure 3.5: Activity Diagram 2 - Static Analysis 46
Figure 3.6: Activity Diagram 3 - Dynamic Analysis 47
Figure 3.7: Activity Diagram 4 – AI/LLM 48
Figure 3.8: Activity Diagram 5 – Report generation 49
Figure 3.9: Activity Diagram 6 – Component fuzzing 50
Figure 3.10: Class diagram 1 52
Figure 3.11: Class diagram 2 53
Figure 3.12: Class diagram 3 53
Figure 3.13: Sequence diagram 1 - User authentication 54
Figure 3.14: Sequence diagram 2- Apk upload 56
Figure 3.15: Sequence diagram 3 - Static analysis 57
Figure 3.16: Sequence diagram 4 - Fuzzing 58
Figure 3.17: Sequence diagram 5 – Dynamic Analysis 60
Figure 3.18: Sequence diagram 6 - AI/LLM reasoning 61
Figure 3.19: Sequence diagram 7 - Report generation 62
Figure 3.20: Sequence diagram 8 – Download PDF 63
Figure 3.21: Sequence diagram 9 - Rescan and report recreation 64

Figure 4.1: Login Screen..... 91
Figure 4.2: Home Dashboard..... 93
Figure 4.3: Upload APK Screen 94
Figure 4.4: Scan Execution Scre..... 95
Figure 4.5: Report Screen 96
Figure 4.6: Scan History Screen 97

List of Tables

Table 1.1: Related System Analysis with proposed project solution..... 18
Table 1.2: Tools and Technologies for Proposed Project..... 19
Table 2.1: User classes in the system..... 22
Table 2.2: Functional Requirements Derived from Mockup M1 25
Table 2.3: Functional Requirements Derived from Mockup M2 26
Table 2.4: Functional Requirements Derived from Mockup M3 28
Table 2.5: Functional Requirements Derived from Mockup M4 30
Table 2.6: Functional Requirements Derived from Mockup M5 31
Table 2.7: Event–Response Table for FalconScan 32
Table 2.8: Software Interfaces 35
Table 2.9: Communication Interfaces..... 35
Table 3.1: Technologies, Components, and Security Mechanisms Used in the System Architecture..... 41
Table 3.2: Key Fields in the FalconScan Scan Results Database 65
Table 4.1: Summary of table API/SDKs..... 89
Table 5.1: Unit test 1 – validate Email 99
Table 5.2: Unit test 2 – check Password Strength 99
Table 5.3: Unit Test - validate APK file..... 100
Table 5.4: Unit test 4 – initiate scan 100
Table 5.5: Functional test 1 - Login with different roles 101
Table 5.6: Functional Test 2 - apk upload 102
Table 5.7: Functional Test 3 - report generation..... 102
Table 5.8: Severity classification..... 103
Table 5.9: Integration Testing 1 - Scan Workflow 104
Table 5.10: Integration Testing 2 - Notification and Report Linking..... 104

1 Chapter 1: Introduction and Problem Definition

This chapter presents a comprehensive overview of the proposed FalconScan project, highlighting the background, motivations, technical foundations, objectives, and the scope of the system. It introduces the problem domain of Android application security, describes the limitations of current solutions, and positions FalconScan as an innovative platform that bridges gaps in mobile cybersecurity. Furthermore, the chapter outlines the modular architecture, technological choices, and expected contributions to the field of mobile application security, demonstrating alignment with academic and practical requirements.

1.1 Overview of the Project

FalconScan is an AI-powered, full-stack Android APK vulnerability scanner designed to automatically identify and report security weaknesses in mobile applications. It combines a hybrid analysis framework—leveraging static code examination, dynamic runtime observation, and AI-based reasoning—to provide a comprehensive understanding of potential vulnerabilities. The system is built as a web-based platform with a Next.js 16 frontend interfacing with a Django 4.2 REST API backend, enabling users to upload APK files, initiate analyses, and receive detailed vulnerability reports in a seamless, automated workflow.

The proliferation of Android applications across industries such as finance, healthcare, e-commerce, and education has made them a prime target for cyber attacks. Despite the widespread use of Android devices, the ecosystem's openness introduces significant security risks, including malware injection, data exfiltration, privilege escalation, and unauthorized access to sensitive resources. Traditional security scanners like MobSF and VirusTotal primarily rely on static, signature-based analyses that are limited to known vulnerabilities. These tools struggle to detect complex, obfuscated, or zero-day threats that may arise only during runtime execution.

FalconScan addresses these limitations by integrating static and dynamic analyses with artificial intelligence for contextual reasoning. During static analysis, the platform inspects the APK structure, metadata, permission declarations, and API call patterns. The dynamic engine then executes the application in a sandboxed environment to monitor runtime behaviors such as

network interactions, file operations, inter-process communication, and system calls. The system aggregates these observations and applies LLM-based reasoning to generate human-readable vulnerability explanations, complete with risk assessments and actionable remediation guidance.

By automating the detection and reporting process, FalconScan significantly reduces reliance on manual intervention, improves accuracy in identifying both known and unknown threats, and equips developers, QA teams, and security analysts with actionable insights. This makes the platform a valuable contribution to mobile cybersecurity, offering a more intelligent, user-friendly, and reliable alternative to conventional tools.

1.2 Vision Statement

FalconScan is envisioned as a next-generation security solution for Android developers, cybersecurity researchers, and organizations seeking to enhance mobile application reliability and protect sensitive user data. As Android applications face growing risks from malicious code, insecure data handling, and inefficient manual vulnerability assessment, FalconScan provides an AI-driven, automated, and comprehensive scanning system.

The platform stands out by combining hybrid analysis techniques with natural language reasoning. Unlike conventional static analyzers that depend solely on predefined signatures and heuristics, FalconScan uses intelligent models to identify and classify vulnerabilities. The system classifies them by severity, explains their technical implications in accessible language, and suggests remediation strategies. Through this approach, FalconScan enables developers and security teams to proactively mitigate risks, streamline vulnerability management, and maintain a higher standard of mobile application security.

1.3 Problem Statement

The security of Android applications continues to be a pressing concern due to the open-source nature of the platform, the sheer volume of applications distributed across official and unofficial marketplaces, and the diverse skill levels of developers. Existing static analysis tools are often inadequate for detecting dynamic threats or obfuscated code, and manual vulnerability testing

demands expert knowledge and extensive effort. As a result, many applications are released with latent security flaws that can be exploited by malicious actors.

Furthermore, static analysis alone is insufficient to detect vulnerabilities that manifest only during runtime, such as dynamic code loading, network-based attacks, or sophisticated evasion techniques. Although dynamic analysis tools exist, they are resource-intensive, slow, and frequently produce unstructured outputs that are difficult to interpret without advanced expertise. Consequently, there is an increasing demand for an integrated platform that automates vulnerability detection, simplifies interpretation of findings, and bridges the gap between static, dynamic, and intelligent reasoning approaches.

1.4 Problem Solution

FalconScan provides a solution by combining hybrid analysis techniques with AI-enhanced reasoning. The system performs comprehensive static analysis, including decompilation, metadata extraction, permission mapping, API pattern analysis, and preliminary security checks. It then executes the APK in a controlled dynamic environment to observe actual runtime behavior, capturing anomalies in file access, network communication, inter-component interactions, and system call sequences.

The integration of a Large Language Model (LLM) enables FalconScan to generate detailed, human-readable explanations of identified vulnerabilities, translating complex technical findings into actionable insights. The platform categorizes vulnerabilities by severity, logs evidence for auditing and compliance, and provides concrete recommendations for remediation. By offering automated scanning, contextual analysis, and clear reporting, FalconScan reduces manual effort, improves detection rates, and empowers developers and security analysts to address security flaws effectively.

Ultimately, FalconScan contributes to a safer Android ecosystem by facilitating early vulnerability detection, enhancing the reliability of mobile applications, and providing an intelligent, adaptive, and user-centric approach to mobile security. Its hybrid, AI-driven methodology represents a significant advancement over traditional tools, ensuring that

developers and organizations can proactively mitigate security risks in their Android applications throughout the software development lifecycle.

1.5 Objectives of the Proposed System

BO-1: Develop a hybrid static and dynamic analysis framework capable of detecting vulnerabilities in Android APKs.

BO-2: Integrate AI/LLM-based reasoning to classify vulnerabilities, provide detailed explanations, and suggest remediation actions for developers and security analysts.

BO-3: Automate the entire APK vulnerability scanning process through an intuitive web interface to minimize manual effort and human error.

BO-4: Maintain a local vulnerability knowledge base to enable rapid detection of common threats and continuous learning from new scans.

BO-5: Enable dynamic runtime monitoring using an Android emulator or sandbox to detect behavioral anomalies and runtime-specific threats.

BO-6: Generate comprehensive, human-readable security reports with severity classifications, evidence, and remediation guidance.

BO-7: Implement a dedicated fuzzing module that tests malformed or boundary-case inputs, identifying potential vulnerabilities in APK components prior to runtime execution.

1.6 Scope

The scope of FalconScan encompasses the complete lifecycle of Android APK security assessment, from secure file submission to comprehensive report generation. Users can upload APKs via a web interface, which automatically triggers modular analysis pipelines. The system's static analysis engine inspects decompiled code, manifest configurations, permission structures,

and embedded components, while the dedicated fuzzing module generates and testss inputs to detect potential vulnerabilities before runtime.

The dynamic analysis engine executes APKs within a controlled Android emulator to capture runtime behaviors, including network requests, file system access, inter-process communication, and system calls. Extracted features from both static, fuzzing, and dynamic modules are processed by a hybrid AI engine that combines machine learning classification and LLM-based reasoning to produce structured vulnerability reports. Reports include categories such as insecure API calls, privilege escalation, sensitive data leaks, and runtime misbehavior, along with severity ratings, textual explanations, and suggested mitigations.

FalconScan is designed as a modular and extensible platform, allowing the integration of additional scanning engines, threat models, or AI capabilities in the future. The platform supports multiple concurrent analyses, secure file handling, and role-based access control for administrators and users. Performance optimization, scalability, and usability are key focus areas, ensuring effective handling of large and complex APKs. The system targets Android developers, QA testers, cybersecurity analysts, and academic researchers in mobile security domains.

1.6.1 Limitations/Constraints

LI-1: FalconScan supports only Android APK files; IOS, hybrid, or other mobile application formats are not covered.

LI-2: Dynamic analysis is limited to virtualized Android emulators; real device testing is not included.

LI-3: LLM-based reasoning may occasionally produce incomplete or imprecise vulnerability explanations due to training data limitations.

LI-4: Certain AI-assisted processing and external vulnerability database updates require internet connectivity.

LI-5: Fuzzing focuses on runtime vulnerability detection but may not identify all runtime-specific edge cases.

LI-6: The system is primarily intended for research, educational, or proof-of-concept use; commercial-grade deployment may require additional enhancements in scalability, performance, and security hardening.

LI-7: Advanced malware reverse engineering, obfuscation removal, and integration with real-time threat intelligence feeds are out of scope for the current version but are planned for future updates.

1.7 Modules

1.7.1 Module 1: User Authentication & APK Management

- **FE-1:** User registration and login via Next.js frontend, backed by Django SimpleJWT auth.
- **FE-2:** Role-based access control (Admin, User, Guest).
- **FE-3:** Upload APKs via web interface, supporting drag-and-drop and multiple files.
- **FE-4:** Validate APK integrity (hash, format) before processing.
- **FE-5:** Store APK metadata (name, size, hash, developer info, timestamps) in Django models.
- **FE-6:** Allow users to delete, archive, or tag APKs.
- **FE-7:** Trigger static and dynamic analysis pipelines automatically upon upload.
- **FE-8:** Notify users of analysis completion via dashboard or toast notifications.

1.7.2 Module 2: Static Analysis Engine

- **FE-1:** Decompile APK using Androguard and JADX.
- **FE-2:** Analyze AndroidManifest.xml for exported components, permission misuse, and intent filters.

- **FE-3:** Detect hardcoded secrets, cryptography misuse, and insecure configurations.
- **FE-4:** Extract static features (permissions, intents, API usage) for AI input.
- **FE-5:** Store static analysis reports and feature vectors for further processing.
- **FE-6:** Provide API endpoint for frontend to retrieve static analysis results.
- **FE-7:** Support rescan functionality for updated APKs.
- **FE-8:** Maintain audit logs of static analysis operations.

1.7.3 Module 3: Fuzzing Engine

- **FE 1:** Automatically launch and interact with APK components on Android emulator using ADB shell commands.
- **FE 2:** Identify fuzzable entry points including exported Activities, Services, Broadcast Receivers, Content Providers, and deep links from parsed manifest data.
- **FE 3:** Generate crafted payloads including malformed extras, oversized strings, and SQL injection patterns targeting component interfaces.
- **FE 4:** Inject fuzzed inputs into running application components via ADB (am start, am broadcast, content query) and Drozer agent when available.
- **FE 5:** Monitor application runtime responses to detect crashes, unhandled exceptions, and anomalous output following fuzz input delivery.
- **FE 6:** Capture and store crash evidence, stack trace fragments, and component-level findings for downstream vulnerability analysis.
- **FE 7:** Escalate to Drozer-based deep probing when available, enabling structured component interaction beyond ADB shell capabilities.
- **FE 8:** Provide structured fuzzing results via a unified findings dictionary including severity breakdown, affected components, and remediation suggestions for AI classification and frontend display.

1.7.4 Module 4: Dynamic Analysis Engine

- FE-1: Launch APKs inside isolated, snapshot-capable sandboxes (emulator/container) with controlled networking and device state rollback.

- FE-2: Instrument and monitor file-system, network (pcap), IPC/Intents, Binder calls, and process activity.
- FE-3: Capture runtime telemetry: system calls, Java exceptions, native crashes, stack traces, and logcat output.
- FE-4: Drive apps with automated input generation (UI events, intents, broadcasts, file/URI injection) and support scripted user flows.
- FE-5: Detect suspicious runtime behaviors (exfiltration, SQL execution from intents, dynamic code loading, privilege escalation, unexpected permissions use).
- FE-6: Extract structured dynamic feature vectors (timelines, syscall counts, network indicators, exception patterns) for ML/LLM input with provenance metadata.
- FE-7: Produce human-readable dynamic analysis logs, summaries, and per-finding evidence (timestamps, traces, captured payloads) for reports.
- FE-8: Archive raw dynamic logs, captures (pcap, traces), and snapshots for retraining, audit, and reproducibility; ensure secure storage and retention policies.

1.7.5 Module 5: LLM-Driven Vulnerability Analysis

- FE-1: Preprocess static, fuzzing, and dynamic features into model-ready inputs (normalisation, tokenisation, vectorisation, provenance tags).
- FE-2: Classify APKs and/or components (benign vs suspicious/malicious) using an LLM-augmented model or ML ensemble, returning labels and scores.
- FE-3: Produce calibrated severity scores and per-finding confidence intervals/probabilities.
- FE-4: Generate concise, evidence-linked natural-language explanations and remediation suggestions for each finding.
- FE-5: Maintain a model lifecycle: monitoring, periodic retraining, dataset versioning, and safe deployment (rollbacks, A/B).
- FE-6: Detect and record false positives/negatives (human-in-the-loop) and feed curated examples back into retraining pipelines.
- FE-7: Correlate and fuse static, fuzzing (dynamic inputs), and runtime findings with provenance to reduce noise and improve accuracy.

- FE-8: Prioritise vulnerabilities by risk context (impact, exploitability, confidence, asset value, exposure) and present actionable ordering.
- FE-9: Expose secure API endpoints for frontend visualization, explainability metadata, audit logs, and model reasoning traces.

1.7.6 Module 6: Reporting & Dashboard UI

- **FE-1:** Next.js/React dashboard displaying APK scan status, history, and statistics.
- **FE-2:** Visualize static, fuzzing, dynamic, and AI analysis results.
- **FE-3:** Export PDF/JSON/CSV reports via Django backend (ReportLab/PyPDF2).
- **FE-4:** Interactive charts (Recharts) and graphs for risk visualization.
- **FE-5:** Display AI reasoning and remediation suggestions alongside vulnerabilities.
- **FE-6:** Toast notifications via Sonner for scan completion or errors.
- **FE-7:** Role-based access control for sensitive actions (delete scan, retrain AI).
- **FE-8:** Filter, sort, and search scan reports dynamically.
- **FE-9:** Mobile-responsive and themeable UI using Tailwind CSS and Vaul theming.

1.8 Related System Analysis/Literature Review

1.8.1 Literature Review

Recent research in Android application security underscores the critical need for comprehensive vulnerability detection that combines both static and dynamic analysis techniques. Static analysis, which examines the code, manifest files, and permission structures without executing the application, is effective for identifying known patterns, insecure API usage, and misconfigurations. However, studies by Faruki et al. (2022) and Wei et al. (2023) have shown that static approaches alone cannot detect vulnerabilities that manifest only during runtime, such as dynamic code loading, network misuse, or obfuscated malicious behaviors. On the other hand, dynamic analysis—executing applications in controlled environments to observe their runtime behavior—provides insight into actual execution paths, resource usage, and inter-component

communication, but it faces challenges in terms of computational overhead, scalability, and handling large or complex APKs.

To overcome these limitations, recent works have explored hybrid analysis frameworks that integrate static and dynamic methods. Hybrid systems can detect a broader range of vulnerabilities by leveraging the strengths of both approaches. For instance, frameworks like MobSF have been widely adopted in research and industry for automated APK analysis, but they primarily rely on rule-based detection methods. These methods are effective against known vulnerabilities but struggle with zero-day threats, obfuscated code, or sophisticated malware variants.

In addition, machine learning and deep learning techniques have gained traction for enhancing detection accuracy. Researchers have employed feature extraction from APKs, permission analysis, API call patterns, and behavioral modeling to train classifiers capable of distinguishing malicious from benign applications. While these approaches improve automation and detection efficiency, they often lack interpretability; they can flag vulnerabilities but fail to explain their origin, impact, or remediation measures in a comprehensible manner. This interpretability gap is a significant challenge, especially for developers and security analysts who require actionable guidance rather than binary verdicts.

Recently, Large Language Models (LLMs) such as GPT and BERT have been explored for their ability to reason about code and natural language. These models can summarize code behavior, analyze potential security risks, and generate human-readable explanations of vulnerabilities, bridging the interpretability gap left by traditional machine learning or rule-based systems. Studies have shown that LLMs can provide context-aware insights, suggest mitigation strategies, and even detect nuanced vulnerabilities that purely automated static or dynamic methods might miss.

FalconScan builds upon this body of research by integrating hybrid static-dynamic analysis with LLM-based reasoning, creating an intelligent and interpretable APK vulnerability scanner. By combining traditional vulnerability detection methods with AI-driven explanations, FalconScan addresses the key limitations of prior systems: it can detect both known and unknown threats,

handle runtime behaviors, and provide clear, actionable guidance for developers, researchers, and QA teams. Furthermore, the inclusion of fuzzing techniques adds an additional layer of runtime vulnerability discovery, enabling the identification of some edge-case input vulnerabilities that may not appear during normal execution.

1.8.2 Related System Analysis

Table 1.1: Related System Analysis with proposed project solution

Application Name	Weakness	Proposed Project Solution
MobSF (Mobile Security Framework)	Limited to rule-based static/dynamic analysis; cannot detect zero-day vulnerabilities; lacks AI reasoning and human-readable explanations.	FalconScan integrates LLM-based reasoning to explain findings in natural language, detects both known and unknown vulnerabilities, and provides detailed remediation guidance.
VirusTotal	Focuses on signature-based malware detection, not comprehensive vulnerability-level analysis; does not include runtime behavior monitoring.	FalconScan performs hybrid analysis combining static, dynamic, and fuzzing methods to detect vulnerabilities at code and runtime levels, including unusual behaviors.
AndroBugs Framework	Offers limited static and dynamic analysis; lacks fuzzing and runtime anomaly detection; minimal contextual reporting.	FalconScan adds sandbox-based dynamic behavior monitoring, fuzzing for edge-case inputs, and interpretable AI-generated reports.
QARK	Detects only common security issues; no machine learning intelligence; lacks detailed vulnerability context and adaptive analysis.	FalconScan uses AI/LLM models for intelligent classification, context-aware vulnerability explanations, severity scoring, and recommended fixes.
Fuzzing Tools (AFL /	Standalone fuzzers cannot provide structured vulnerability reports or	FalconScan integrates fuzzing into the hybrid analysis pipeline, automatically

libFuzzer)	integrate runtime behavior context; purely input-focused.	generating mutated inputs, testing APK components, and reporting anomalies with explanations.

1.9 Tools and Technologies

Table 1.2: Tools and Technologies for Proposed Project

Tools and Technologies	**Purpose**	**Rationale**
Python (Django + DRF)	Backend APIs, APK analysis, reporting	Robust, scalable, and widely used for AI-integrated web applications.
Next.js 16 + React 18	Frontend development, dashboard UI	Component-based, fast, interactive, supports modern TypeScript features.
Tailwind CSS, Radix UI, Recharts	UI styling, components, data visualization	Provides responsive, accessible, and visually appealing dashboards.
Axios, Sonner, cmdk	API calls, notifications, command palette	Simplifies REST communication and improves UX.
TensorFlow / PyTorch (optional)	AI model training and inference	Enables machine learning-based vulnerability classification and reasoning.
Droozer	Fuzzing	Automatically tests malformed/boundary-case inputs to detect robustness issues.
LLM Integration	Natural language vulnerability explanation	Provides interpretable, context-aware reasoning for vulnerabilities.
SQLite	Local database	Lightweight, sufficient for user data and scan metadata; easily extendable.
Android Emulator (AVD)	Dynamic analysis sandbox	Safely monitors runtime app behaviors without real device risk.
Docker (optional)	Deployment and environment management	Ensures reproducibility and isolation across development and production.
ReportLab / PyPDF2	PDF report generation	Generates structured, readable security

		reports for users.

1.10 Project Contribution

The FalconScan system introduces several technical and conceptual innovations that advance the state-of-the-art in Android application security analysis. Unlike existing solutions, which primarily rely on static or rule-based scanning, FalconScan combines hybrid analysis techniques, AI-driven reasoning, and automated reporting to provide a comprehensive and user-friendly security platform. The project's contributions span multiple dimensions, including innovation, technical design, usability, and integration of modern tools.

Key Contributions:

- **AI-Enhanced Vulnerability Detection:** Implements machine learning classifiers capable of identifying both known and previously unseen vulnerabilities, including obfuscated or zero-day threats, going beyond traditional signature-based detection.
- **LLM-Based Explainability:** Integrates large language models to generate human-readable explanations of identified vulnerabilities, detailing potential impact, root cause, and recommended remediation steps.
- **Hybrid Static and Dynamic Analysis:** Combines static code and manifest analysis with dynamic runtime monitoring in a sandboxed Android emulator, providing comprehensive coverage of potential attack vectors.
- **Fuzzing Integration:** Introduces automated fuzz testing within static analysis, generating malformed or boundary-case inputs to detect latent vulnerabilities before runtime execution.
- **Automated End-to-End Workflow:** Fully automates the scan-to-report pipeline, from APK upload to detailed PDF report generation, significantly reducing manual effort and human error.
- **Extensible and Modular Architecture:** Designed to support future integration of additional analysis engines, threat models, AI classifiers, and reporting formats, enabling continuous improvement and scalability.

- **Real-Time and Interactive Reporting:** Provides immediate feedback via an intuitive web dashboard with graphical visualizations, detailed tables, and downloadable reports for developers, QA teams, and security researchers.
- **Secure and Efficient Execution:** Implements role-based access control, sandboxed analysis, and encrypted data handling to maintain operational security and user trust.

Impact:

By integrating AI reasoning, hybrid analysis, and automated workflows, FalconScan transforms traditional vulnerability scanning into a proactive, intelligent, and interpretable platform. It improves detection accuracy, reduces the reliance on expert intervention, and enhances mobile application resilience. Developers and security analysts benefit from clear guidance, actionable insights, and operational efficiency, while the modular design ensures that the system can evolve with emerging threats and modern Android app complexities.

1.11 Relevance to Course Modules

- **Software Engineering Concepts:** Applied full SDLC phases, including requirement analysis, system design, implementation, and testing.
- **Artificial Intelligence / Machine Learning:** Integrated ML classifiers and LLMs for intelligent APK vulnerability detection, classification, and explanation.
- **Database Systems:** Designed structured storage for users, scans, and vulnerability data using SQLite (or PostgreSQL for larger deployments).
- **Web Technologies:** Built full-stack web application using Next.js + React for frontend and Django for backend APIs and services.
- **Information Security:** Implemented secure JWT authentication, sandboxed Android emulator execution, static/dynamic analysis, and data encryption.
- **Cloud Computing / DevOps:** Used Docker to ensure environment consistency, reproducible deployment, and scalable execution.

2 Chapter 2: Requirement Analysis

This chapter provides a detailed analysis of the requirements for the FalconScan system. The

purpose of this chapter is to define and understand what the proposed software must accomplish in terms of both functionality and performance. It outlines the user classes and their interactions, the system's functional and non-functional requirements, and its external interface needs. This requirement analysis forms the foundation for subsequent design, development, and implementation phases of FalconScan – an AI-powered APK Vulnerability Scanner that intelligently analyzes Android applications for potential security risks through both static and dynamic analysis.

2.1 User Classes and Characteristics

FalconScan is designed for a variety of users ranging from developers to security professionals. Each user class interacts with the system differently depending on their role, responsibilities, and level of technical expertise. The following table defines each identified user class and its pertinent characteristics.

Table 2.1: User classes in the system

User Class	Description
Application Developer	The primary user who uploads Android APK files to scan for vulnerabilities. Developers use FalconScan to identify and fix potential threats such as insecure permissions, data leaks, or embedded malware. They have access to scan reports, vulnerability breakdowns, and AI-generated recommendations.
Security Analyst	A user who performs detailed analysis on scan reports and verifies AI-generated findings. Analysts may also use the system to test known malware or suspicious apps in a controlled environment to validate FalconScan's detection performance.
Administrator	The system administrator manages user accounts, controls system configurations, and monitors scanning operations and logs. Administrators have full privileges to update the AI model, manage API keys, and ensure smooth operation of the system.
Researcher	Users involved in cybersecurity research who use FalconScan's dataset and analytical outputs for vulnerability pattern studies. Researchers interact with scan results and statistical summaries but do not modify core system settings.

Guest User	A limited-access user who can perform demo scans or view general information about FalconScan without registration. This class is mainly intended for showcasing the platform's capabilities to potential users or evaluators.

2.1.1 Use case Diagram

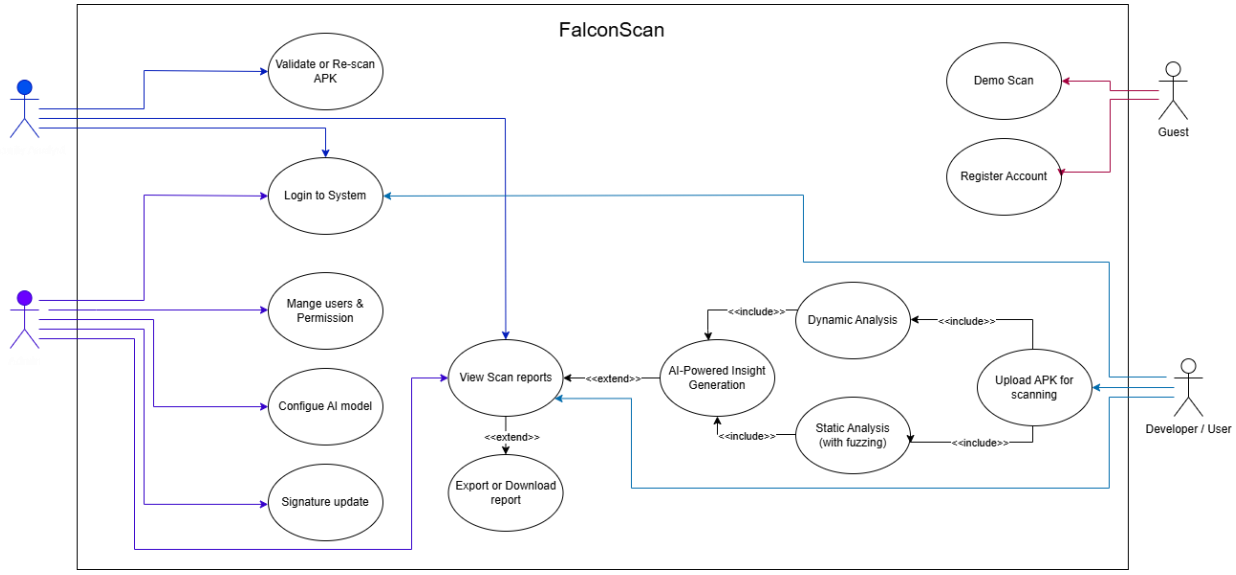


Figure 2.1: Use case diagram

2.1.2 Requirement Identifying Technique

For this project, **Mockup-Based Requirement Analysis** is used. This approach focuses on deriving functional requirements directly from the designed user interface mockups. Each mockup corresponds to a specific user interaction screen in FalconScan.

Steps Followed

1. Identified key features of FalconScan and represented each through a corresponding mockup.

2. Assigned Mockup IDs:
3. Created UI mockups that visualize each screen's layout and features.
4. Identified interactive elements such as buttons, upload fields, report cards, and filters.
5. Derived functional requirements based on the expected system behavior when these elements are used.
6. Organized all requirements using a tabular format (as shown in Section 2.2.1).

Backend Process Requirements

While the mockup-based technique effectively captures frontend behavior, FalconScan also includes extensive backend processes for file analysis, AI-based classification, and report generation.
For these, the Event–Response Table technique is used to define system behavior based on specific backend triggers and events.

Examples include:

- Handling file uploads and initiating analysis workflows.
- Running AI-based vulnerability detection models.
- Generating reports and storing historical scan data.

2.2 Functional Requirements

This section documents all the functional requirements derived from FalconScan's user interface mockups (M1–M5) and backend processes. Each requirement is defined clearly, testably, and traceably.

2.2.1 Mockup M1 – Login Page

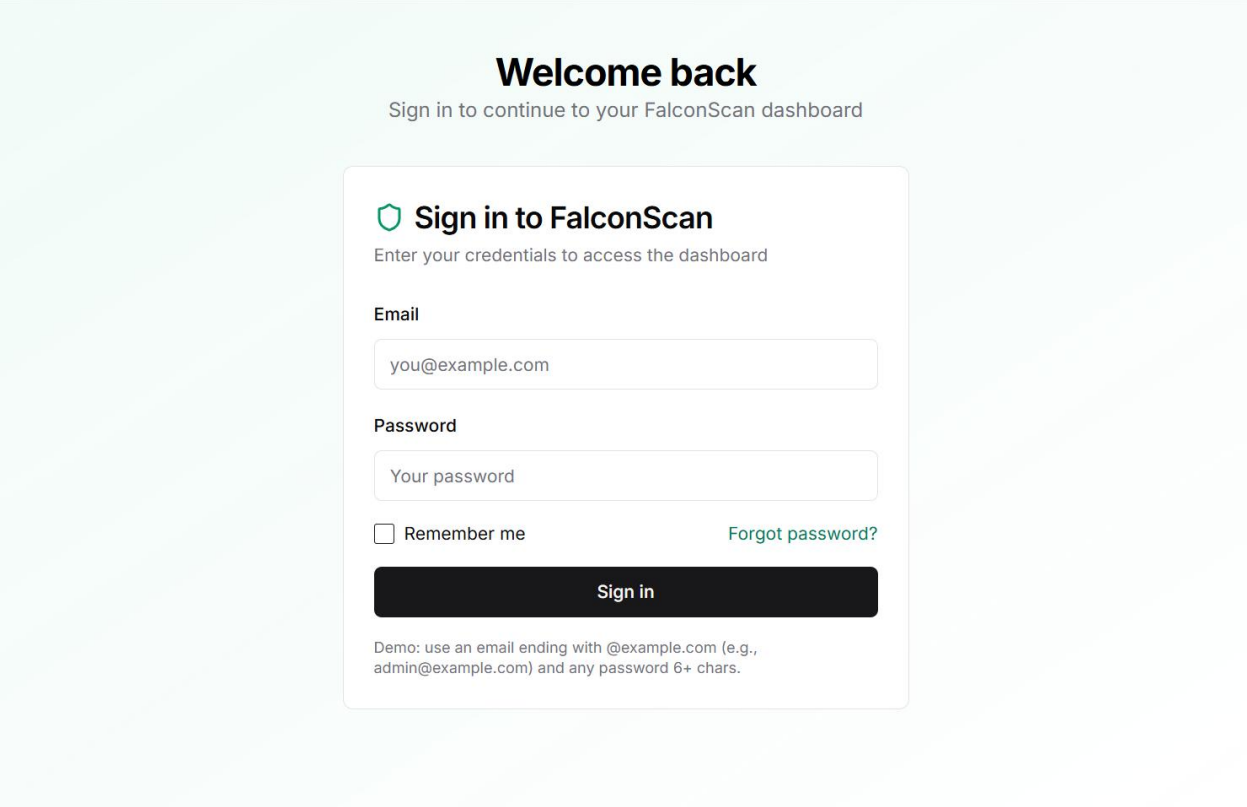


Figure 2.2: Login page

Functional Requirements Derived from Mockup M1 – Login Page

Table 2.2: Functional Requirements Derived from Mockup M1

Feature	Functional Requirement	Business Rule
User Sign-In	**FR1.1:** System shall allow users to log in using valid email and password credentials.	Email must follow RFC 5322 format; Password must be ≥ 8 characters.
Credential Validation	**FR1.2:** System shall validate user credentials against the authentication database.	Accounts locked after 5 failed attempts.
Password Recovery	**FR1.3:** System shall provide a "Forgot Password" option for verified users.	OTP or reset link expires after 10 minutes.
Session	**FR1.4:** System shall maintain user sessions	Token expires after 30 minutes

Management	using secure JWT tokens.	of inactivity.
Role-Based Redirect	**FR1.5:** System shall redirect users to pages based on role (Admin/Analyst/Standard User).	Only Admins see "Model Management" and "User Management".

2.2.2 Mockup M2 – Dashboard Page

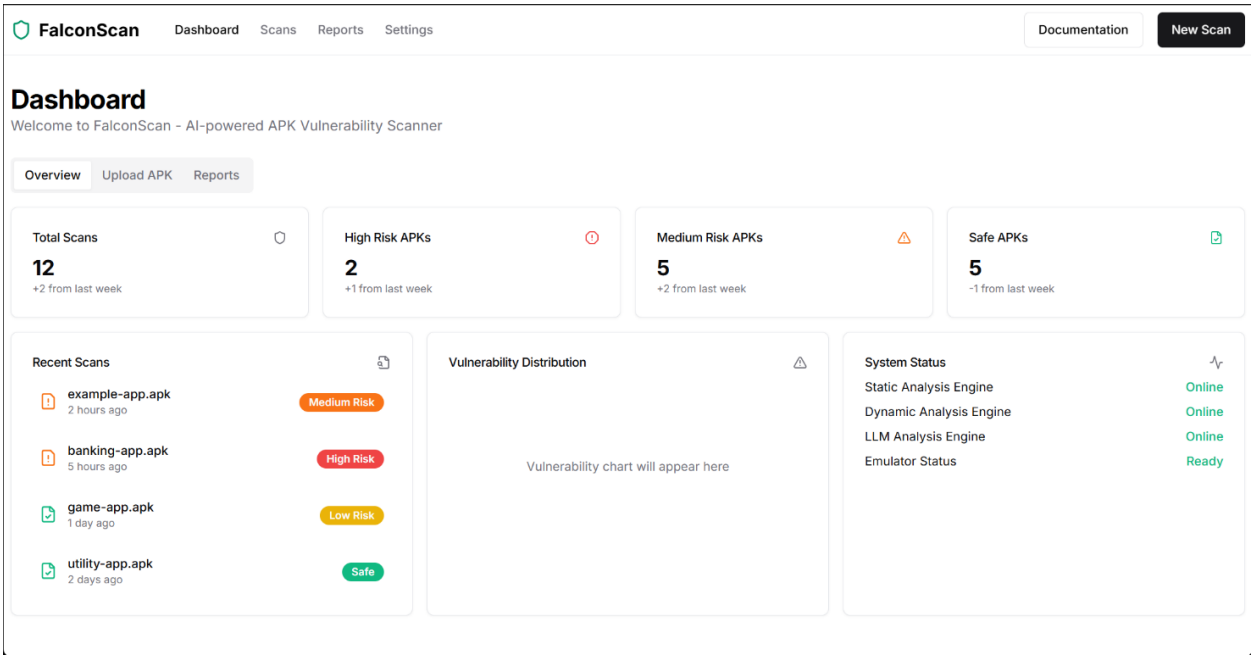


Figure 2.3: Dashboard Page

Functional Requirements Derived from Mockup M2 – Dashboard Page

Table 2.3: Functional Requirements Derived from Mockup M2

Feature	Functional Requirement	Business Rule
User Overview	**FR2.1:** System shall display user profile information and scan statistics.	Data refreshes every 5 seconds.
Navigation	**FR2.2:** System shall provide navigation to Upload, Reports, History, Settings, and Admin Panel (if applicable).	Menu items depend on user role.
Dashboard Widgets	**FR2.3:** System shall show widgets: total scans, vulnerabilities detected, AI confidence, last	Widgets update dynamically from

	activity.	analytics engine.
Recent Activity Log	**FR2.4:** System shall show last 10 system activities for the logged-in user.	Logs are stored for 90 days.
System Notifications	**FR2.5:** System shall display notifications such as “Scan Completed”, “Model Updated”, etc.	Notifications auto-expire after 7 days.
Admin Analytics	**FR2.6:** System shall show global analytics only to Admin (total platform scans, trending vulnerabilities).	Accessible only with Admin role.
Dark/Light Theme Toggle	**FR2.7:** System shall support UI theme toggling.	User preference saved in profile settings.

2.2.3 Mockup M3 – Upload APK Page

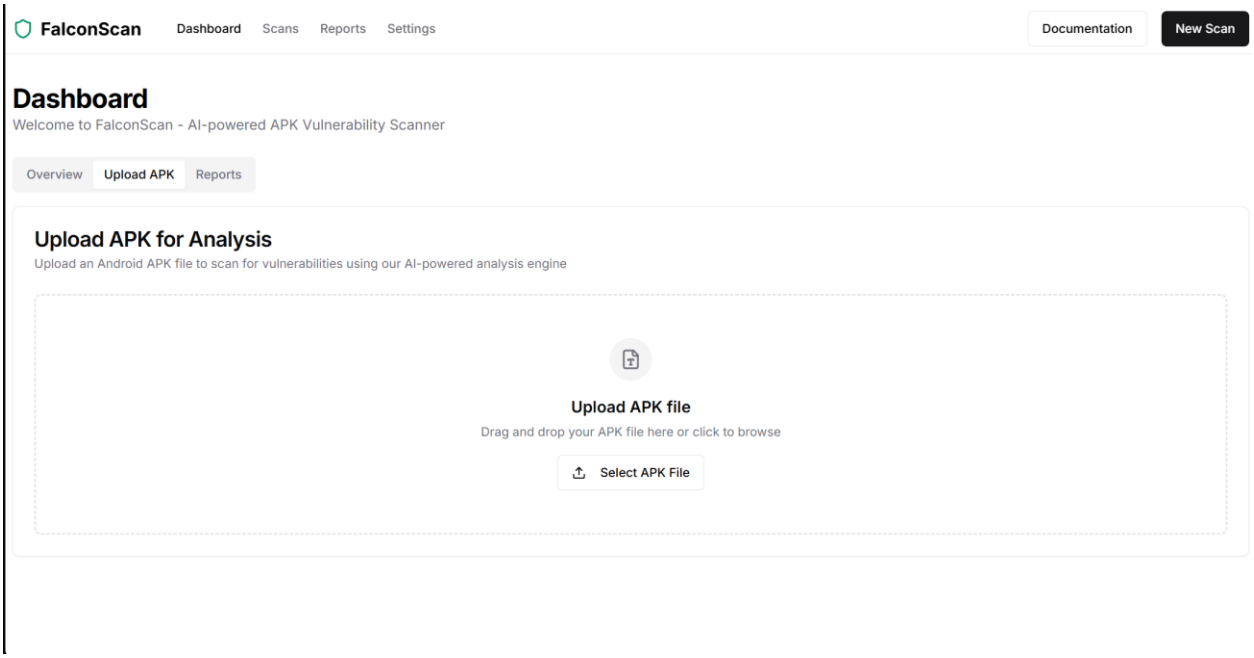


Figure 2.4: Upload APK Page

Functional Requirements Derived from Mockup M3 – Upload APK Page

Table 2.4: Functional Requirements Derived from Mockup M3

Feature	Functional Requirement	Business Rule
APK Upload	**FR3.1:** System shall allow upload of APK files for analysis.	Max size 200 MB. Only `.apk` allowed.
Automated Analysis	**FR3.2:** System shall automatically trigger Static & Dynamic analysis after upload.	No analysis begins if file signature is invalid.
Scan Progress	**FR3.3:** System shall show real-time scan progress bars for each stage.	User can cancel scan only before "Analysis Phase".
Access Control	**FR3.4:** System shall restrict uploads to authenticated users.	Guest users can only run demo scans with sample APK.
Fuzzing Configuration	**FR3.5:** System shall allow advanced users to configure fuzzing options such as input size, execution cycles, and target components.	Only Admins/Analysts have access to advanced fuzz settings.
File Validation	**FR3.6:** System shall validate file integrity using checksums before analysis.	Reject corrupted or incomplete APKs.
Duplicate Scan Handling	**FR3.7:** If a file has been scanned before, system shows past results and offers "Re-scan".	Hash-based duplicate detection.
Upload History Preview	**FR3.8:** System shall show last 5 uploaded APKs as shortcuts.	Visible only to current user.
Dependency Extraction	**FR3.9:** System shall perform extraction of app metadata (permissions, activities, libraries).	Must complete before static analysis stage.
Scan Queue Management	**FR3.10:** If server is busy, system shall place uploads into queue.	Queue position displayed to the user.

2.2.4 Mockup M4 – Scan Reports Page

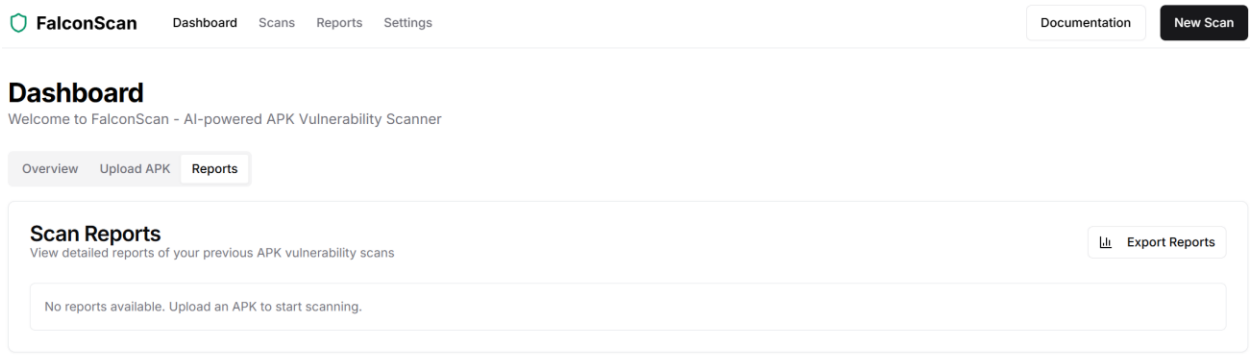


Figure 2.5: Scan Reports Page

Functional Requirements Derived from Mockup M4 – Scan Reports Page

Table 2.5: Functional Requirements Derived from Mockup M4

Feature	Functional Requirement	Business Rule
Vulnerability Display	**FR4.1:** System shall display all vulnerabilities categorized by severity.	Severity follows CVSS 3.1 metrics.
AI Explanations	**FR4.2:** System shall generate AI explanations for each finding.	Must be concise, technical, and accurate.
Report Export	**FR4.3:** System shall export reports in PDF, JSON, and CSV (NEW).	Only owner or Admin can export.
False Positive Marking	**FR4.4:** System shall allow users to mark vulnerabilities as verified or false-positive.	Feedback updates AI training pipeline.
Technical Remediation	**FR4.5:** System shall provide fix recommendations for each vulnerability.	Recommendations follow OWASP MASVS.
Code Snippet Extraction	**FR4.6:** System shall highlight decompiled code segments linked to vulnerabilities.	Only static findings produce snippets.
Behaviour Timeline	**FR4.7:** System shall show dynamic behavior timeline (network calls, I/O, permissions).	Based on dynamic sandbox logs.
AI Confidence Score	**FR4.8:** System shall display AI confidence % for each finding.	Scores must be between 0–100%.
Risk Summary Chart	**FR4.9:** System shall show graphical charts (severity distribution, module-level risk).	Charts updated in real time.

2.2.5 Mockup M5 – Scan History Page

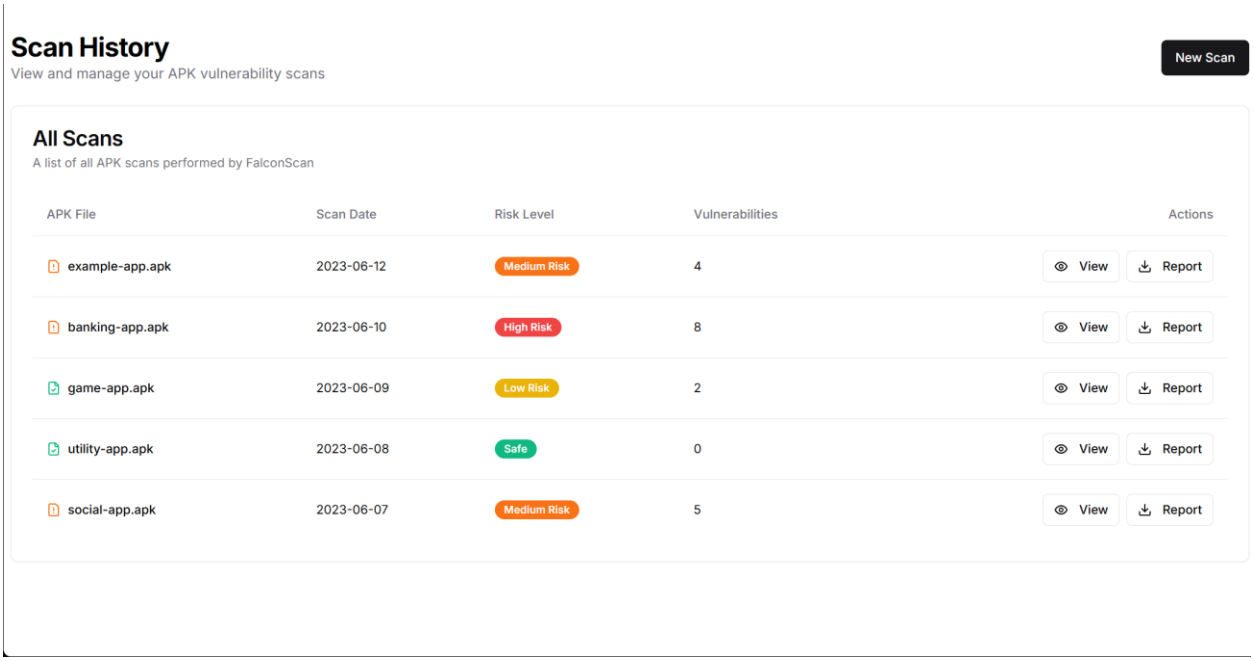


Figure 2.6: Scan History Page

Functional Requirements Derived from Mockup M5 – Scan History Page

Table 2.6: Functional Requirements Derived from Mockup M5

Feature	Functional Requirement	Business Rule
Scan Records	**FR5.1:** System shall maintain chronological scan history.	Includes date, filename, hash, size, and status.
Filtering	**FR5.2:** System shall allow filtering by date, severity level, scan type, or result.	Max 100 records per page.
Rescan	**FR5.3:** System shall allow re-scanning past files.	Only original uploader can re-scan.
Delete Record	**FR5.4:** System shall allow users to delete their own scan history entries.	Irreversible once deleted.
Bulk Actions	**FR5.5:** System shall support bulk export or bulk delete.	Maximum 20 selections at a time.
Search Function	**FR5.6:** System shall provide search by filename, hash, or result.	Search supports substring matching.

Archive Old Scans	**FR5.7:** System shall automatically archive scans older than 90 days.	Archived scans stored in cold storage.

2.2.6 Event–Response Table for Backend Processes

Table 2.7: Event–Response Table for FalconScan

Event	System State	System Response
User uploads APK	File received	System verifies file signature → stores temporary copy → adds job to queue.
Static analyzer detects insecure permission	Static analysis phase	System logs finding → assigns severity → stores metadata.
Static analyzer finds obfuscated code	Static analysis phase	System flags potential malware → sends data to AI model.
Dynamic analyzer detects suspicious network request	Runtime sandboxing	System stores PCAP logs → extracts endpoints → adds to report.
Fuzzing analysis initiated	Fuzzer active	System generates test payloads → executes inside instrumented environment.
Fuzzing detects abnormal behavior	Fuzzer runtime	System logs crash, stack traces, memory corruption patterns.
AI model completes classification	Post-analysis stage	Generates predictions → sorts vulnerabilities → generates explanations.
Model fails to classify	AI timeout/error	System triggers fallback rule-based detection mechanism.
User requests report export	Ready for export	System formats file into PDF/JSON/CSV and triggers download.
Admin updates AI model	Model management module	System archives old model → deploys new version → logs update.
Scan queue overloaded	Queue threshold exceeded	System notifies user and provides estimated wait time.
Database connection failure	Error handler	System retries operation 3 times → logs

	active	error → notifies Admin.
Sandbox detects evasion attempt	Dynamic execution	System flags sample as evasive → increases severity score.

2.3 Non-Functional Requirements

2.3.1 Reliability

- **REL-1:** The system shall maintain a minimum uptime of 99.5% under normal operating conditions.
- **REL-2:** The system shall have a Mean Time Between Failures (MTBF) of at least 500 **hours**.
- **REL-3:** All scan progress shall be auto-saved every 30 seconds to prevent data loss during system failure.
- **REL-4:** In the event of an analysis crash, the system shall resume from the last saved state.

2.3.2 Usability

- **USE-1:** The system shall allow first-time users to complete a full scan process (upload → report) within 3 intuitive clicks**.**
- **USE-2:** The user interface shall maintain consistent color schemes, icons, and typography across all pages.
- **USE-3:** Tooltips and help sections shall be available for all major features.
- **USE-4:** The platform shall be accessible via both desktop and mobile browsers with responsive design.

2.3.3 Performance

- **PER-1:** 90% of scans for APKs under 100 MB shall complete within 3 minutes on standard cloud infrastructure.

- **PER-2:** 95% of pages shall load fully within 4 seconds over a 20 Mbps connection.
- **PER-3:** Concurrent scanning sessions shall support at least 50 simultaneous users without degradation.

2.3.4 Security

- **SEC-1:** The system shall use HTTPS for all communications between client and server.
- **SEC-2:** Uploaded files shall be isolated in sandboxed environments to prevent system compromise.
- **SEC-3:** All user data shall be encrypted in transit and at rest using AES-256.
- **SEC-4:** The system shall automatically log all access attempts and notify the admin of failed authentication exceeding 5 consecutive attempts.
- **SEC-5:** The AI model training data shall be anonymized and stored securely.

2.4 External Interface Requirements

2.4.1 User Interface Requirements

The FalconScan UI shall follow a clean, consistent design standard based on Material UI principles.
Key UI requirements include:

- Standardized color palette: dark theme with accent highlights.
- Font: Open Sans or Roboto, consistent across all screens.
- Every page shall include a top navigation bar with Home, Dashboard, Upload, Reports, and History links.
- Common buttons such as “Scan”, “Export”, and “Logout” will be present on applicable screens.
- All error messages will appear in red banners at the top of the interface with clear descriptions.
- A help icon (“?”) will appear on each screen for guidance.

- The interface shall support accessibility features such as screen reader compatibility and high-contrast mode.

2.4.2 Software Interfaces

Table 2.8: Software Interfaces

Interface ID	Description
SI-1: AI Model API	The system shall communicate with a Python-based vulnerability detection API for AI processing.
SI-2: Database Interface	The system shall interact with a SQLite database for user, scan, and report management.
SI-3: Emulator Service	The system shall interface with an Android emulator API to perform dynamic runtime analysis.
SI-4: Report Generator Module	The system shall generate and export reports via an integrated Python report generation library.

2.4.3 Hardware Interfaces

FalconScan runs primarily on device and requires access to specific hardware during analysis:

- Android emulator host machines with at least 8 GB RAM and 4 CPU cores for dynamic testing.
- Secure storage devices for maintaining uploaded APKs and scan results.

2.4.4 Communication Interfaces

Table 2.9: Communication Interfaces

Communication ID	Requirement
CI-1:	The system shall communicate over HTTPS using TLS 1.3 protocol.
CI-2:	The system shall send email notifications to users upon completion of scans.

CI-3:	The system shall allow API communication for integration with third-party tools.
CI-4:	The system shall log all internal communications for traceability and debugging.

3 Chapter 3: Design and Architecture

This chapter presents the complete architectural design of the proposed FalconScan Android APK Vulnerability Analysis System. It covers the conceptual system architecture, architectural patterns, design models, data representations, and the rationale behind the architectural decisions taken during the system's development. The chapter also explains how the different system modules interact with each other to form a cohesive, secure, scalable, and extensible full-stack security analysis platform.

FalconScan uses a Hybrid Architecture that combines the strengths of three major architectural approaches:

1. **Client–Server Architecture**
   Used for communication between the Next.js frontend and the Django REST API server.
2. **Layered Architecture**
   Used within the backend to separate presentation, business logic, analysis engines, AI reasoning, and database layers.
3. **Modular Pipeline Architecture**
   Used in the analysis engine where each module (static analysis, dynamic analysis, fuzzing, AI reasoning, reporting) works independently, is replaceable, and can be extended without affecting other components.

This hybrid approach ensures scalability, maintainability, high system clarity, and extensibility as the APK scanning logic grows over time.

3.1 System Architecture Overview

This section provides the high-level external architectural design of the FalconScan system. It includes the major components, technologies, communication flow, and security controls used across the system. Unlike the internal architecture patterns (Layered, MVC, etc.), this section focuses on the **macroscopic view** to help stakeholders understand how the entire system behaves as a whole.

FalconScan is structured as a cloud-enabled, full-stack APK scanning and reporting platform accessed through a web-based dashboard. Users interact with the system through the Next.js 16 web client, which communicates with the Django REST Framework backend responsible for authenticating users, managing scans, executing static and dynamic analysis, performing AI-based reasoning, and generating PDF reports.

Uploaded APKs are securely stored under the backend's media repository and passed through multiple hybrid analysis stages, including manifest parsing, permission analysis, opcode inspection, string extraction, anomaly detection, LLM-based reasoning, and PDF report generation. All results, metadata, and analytic insights are stored in the relational database.

3.1.1 Purpose

The purpose of this section is to illustrate a conceptual overview of FalconScan. It presents the big-picture structure of the system and identifies major functional modules, data flows, and high-level communication pathways. This helps stakeholders understand:

- What the system contains
- Where each module resides
- How subsystems communicate
- Which technologies are used
- How data and control signals move across components
- What security defenses protect the system

This conceptual view serves as a foundation for understanding deeper architectural decisions presented later in the chapter.

3.2 Guidelines for Students

The conceptual architecture diagram visually depicts the core modules of FalconScan and the relationships between them. It identifies the frontend user interface, backend services, authentication subsystem, database storage, analysis pipeline, admin panel, and external integrations. Arrows represent data flow and control signals between components.

3.2.1 Conceptual Architecture Diagram

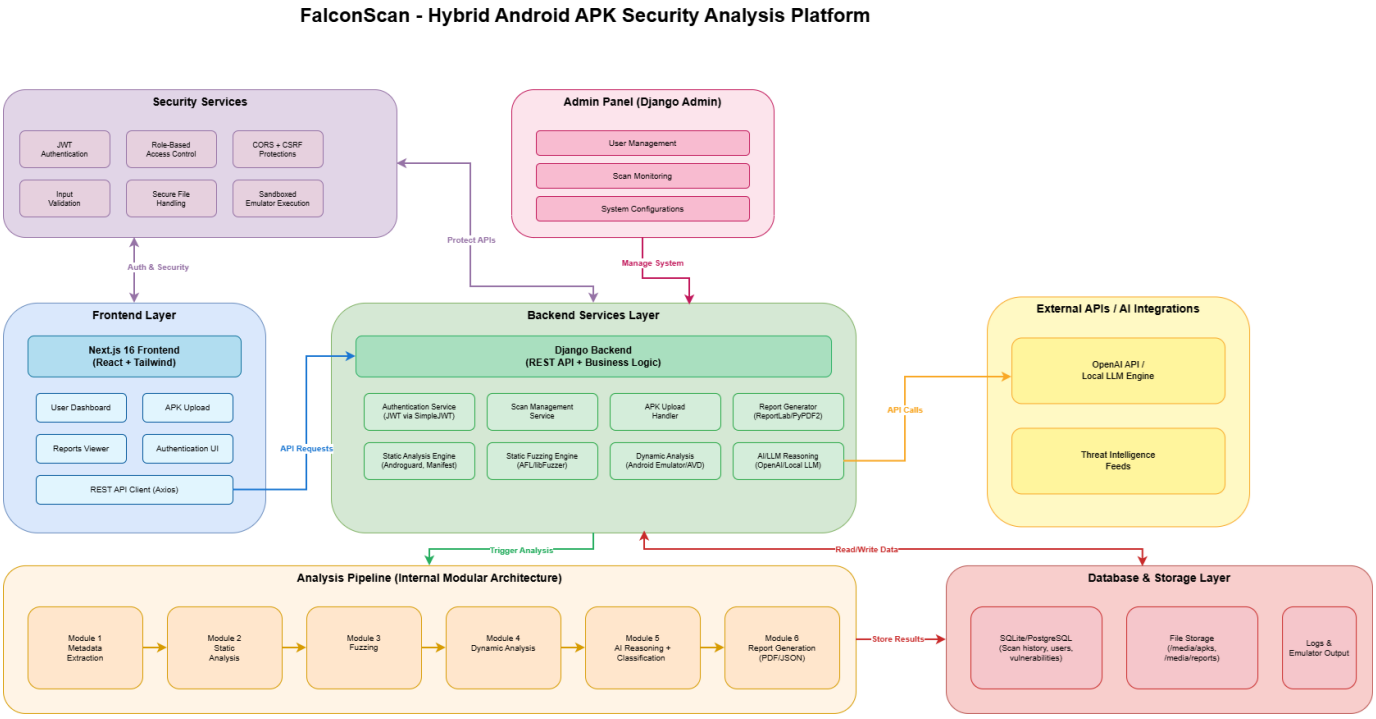


Figure 3.1: Conceptual Architecture diagram

3.2.1.1 Frontend (Next.js / React UI)

The frontend is the primary interaction layer for end-users. It provides authentication pages, APK upload interface, dashboard visualizations, scan history pages, detailed vulnerability reports, and administrator functionalities. The interface is responsive, modular, and optimized for high-performance rendering using React components.

3.2.1.2 Backend Services (Django REST API)

This is the core processing unit of the entire system. It manages authentication, user sessions, scan management, report generation, APK processing, and database interactions. It exposes secure RESTful endpoints consumed by the frontend.

3.2.1.3 Database Layer (SQLite)

Stores users, scans, vulnerabilities, metadata, system configurations, and generated reports. The relational model ensures structured, normalized, and indexed data suitable for analytics-heavy workflows.

3.2.1.4 Authentication & Authorization Service

Implements JWT authentication via SimpleJWT. Handles user login, access tokens, refresh tokens, role-based access control (RBAC), and secure session management.

3.2.1.5 Analysis Pipeline

A modular, extensible architecture enabling independent scanning components:

- Static Analysis Module (permissions, manifest, opcode, strings)
- Dynamic Analysis Module (sandbox execution, log capture)
- AI Reasoning Module (LLM vulnerability explanation & scoring)
- Fuzzing Module
- Reporting Module (PDF generation)

Each module can be replaced or upgraded without affecting others.

3.2.1.6 Admin Panel

Allows administrators to delete scans, monitor platform usage, manage users, view logs, and maintain system configurations.

3.2.1.7 Security Services

- JWT token-based authentication
- Input validation
- File safety checks

- RBAC policies
- CORS protection
- Internal sandbox for dynamic analysis

3.2.2 Technologies and Services

Table 3.1: Technologies, Components, and Security Mechanisms Used in the System Architecture

Component	Description	Technology Used	Security Mechanism
Frontend App	User-facing interface for APK uploads, dashboards, and report viewing	Next.js 16, React 18, Tailwind CSS	SSL/TLS, Input Validation, Secure Storage
Backend API	Business logic, scan processing, and communication with analysis modules	Django 4.2, Django REST Framework	JWT, Role-Based Access Control (RBAC), CORS Protection
Database	Structured data storage for users, scans, vulnerabilities, reports	SQLite (development)	RBAC, Encrypted Storage (optional), ORM-level input sanitization
Authentication Service	Token-based identity and session management	SimpleJWT (Django)	Access Tokens, Refresh Tokens, Expiration Policies
Analysis Engine	Hybrid multi-stage APK analysis modules	Androguard, PyAXMLParser, ADB-based dynamic analysis, Python ML	Sandbox Isolation, File Validation
Admin Panel	Administrative management dashboard	React (within Next.js)	Admin Authorization, Audit Logs
AI Reasoning Service	LLM-based vulnerability	Integrated LLM APIs	Secure API Key Management

	interpretation and report enrichment		
Reporting Engine	Automated PDF report generation	ReportLab, PyPDF2	Integrity Validation, Secure File Storage

3.3Architecture Style / Pattern

FalconScan uses a **Hybrid Architecture Pattern** that integrates:

- **Client–Server Architecture**
- **Layered Architecture**
- **Modular Pipeline Architecture**

This combination allows the system to be scalable, extensible, and maintainable while supporting complex APK analysis operations.

3.3.1 Selected Architecture Pattern & Justification

The FalconScan system uses a combination of architectural patterns that work together to ensure scalability, maintainability, and high performance APK analysis. At the highest level, the platform follows a Client Server Architecture, where the client application (web dashboard) communicates with a centralized backend server through secure HTTP APIs. This separation allows the client to remain lightweight and focused on user interaction while the backend handles all heavy processing tasks such as static analysis, dynamic execution, fuzzing, and AI driven vulnerability classification. The client server model is ideal here because it ensures a secure communication channel, supports authenticated user access, and keeps all sensitive APK processing inside a controlled server environment rather than exposing it on the client side.

Inside the backend, the system applies a Layered Architecture, dividing the server into well defined layers: the Presentation Layer exposes REST API endpoints for authentication, APK upload, and report retrieval; the Business Logic Layer manages user workflows, orchestrates scan operations, and enforces application rules; the Scanning and AI Processing Layer contains

specialized engines responsible for static analysis, dynamic behavior monitoring, fuzzing, and AI reasoning; the Data Access Layer (DAL) uses Django ORM repositories to abstract database operations; and the Storage Layer handles the physical storage of APK files, logs, scan results, and generated reports. This layered design reduces coupling between components, making the system easier to extend and maintain. Updates to one layer do not affect others as long as the interfaces remain consistent.

The scanning engine itself follows a Modular Pipeline Architecture, where analysis tasks are broken into modular, sequential stages. Each module Static Analysis, Dynamic Analysis, Fuzzing, AI Interpretation, and Reporting operates independently and communicates through structured intermediate results. This modularity ensures that any component can be extended, improved, or even replaced, such as upgrading the static analyzer or swapping the AI model, without disrupting the rest of the pipeline. The pipeline model allows parallel development, simplifies debugging, and ensures that the overall scanning process is systematic, reproducible, and optimized for handling large numbers of APKs. Overall, the combination of these three architectural patterns provides a robust, scalable, and secure foundation for FalconScan's APK vulnerability scanning workflow.

3.3.2 Architecture Style Diagrams

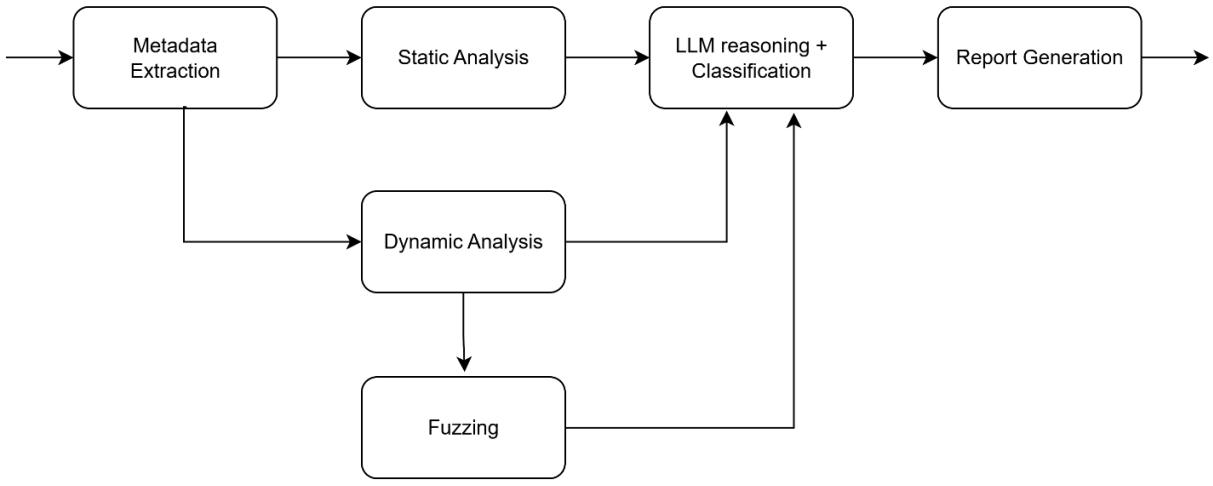


Figure 3.2: Pipeline Architecture

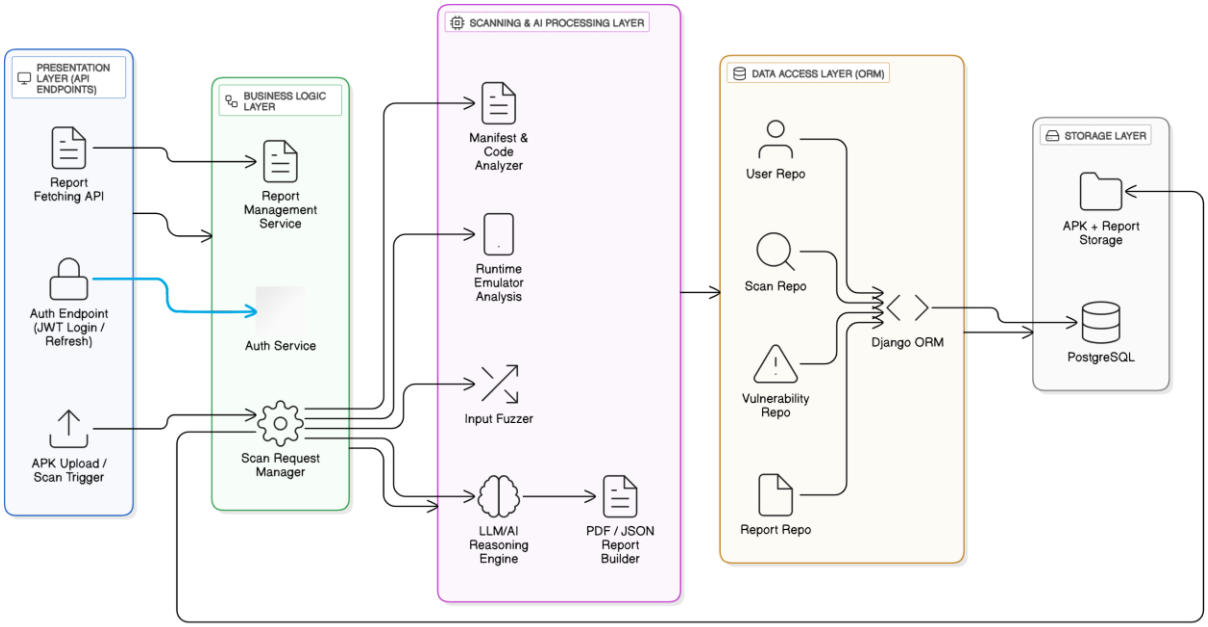


Figure 3.3: Layered Architecture inside the backend

3.4 Design Models

This section provides a comprehensive overview of the key design models used in FalconScan, an Android APK security platform. These models are developed using the object-oriented development (OOD) approach, which emphasizes modularity, encapsulation, inheritance, and reusability. The design models included here focus on core workflows, important system components, and critical processes that are central to FalconScan's functionality, rather than trivial operations such as login/logout or simple CRUD actions.

Object-oriented design models help in visualizing the internal structure of the system, understanding the relationships between classes, and clarifying the flow of control and data among objects. By using these models, developers, testers, and stakeholders can grasp the architecture, behavior, and responsibilities of major system components, facilitating maintenance, extension, and collaboration.

3.4.1 Design Models for Object Oriented Development Approach

3.4.1.1 Activity Diagram

1. Apk upload

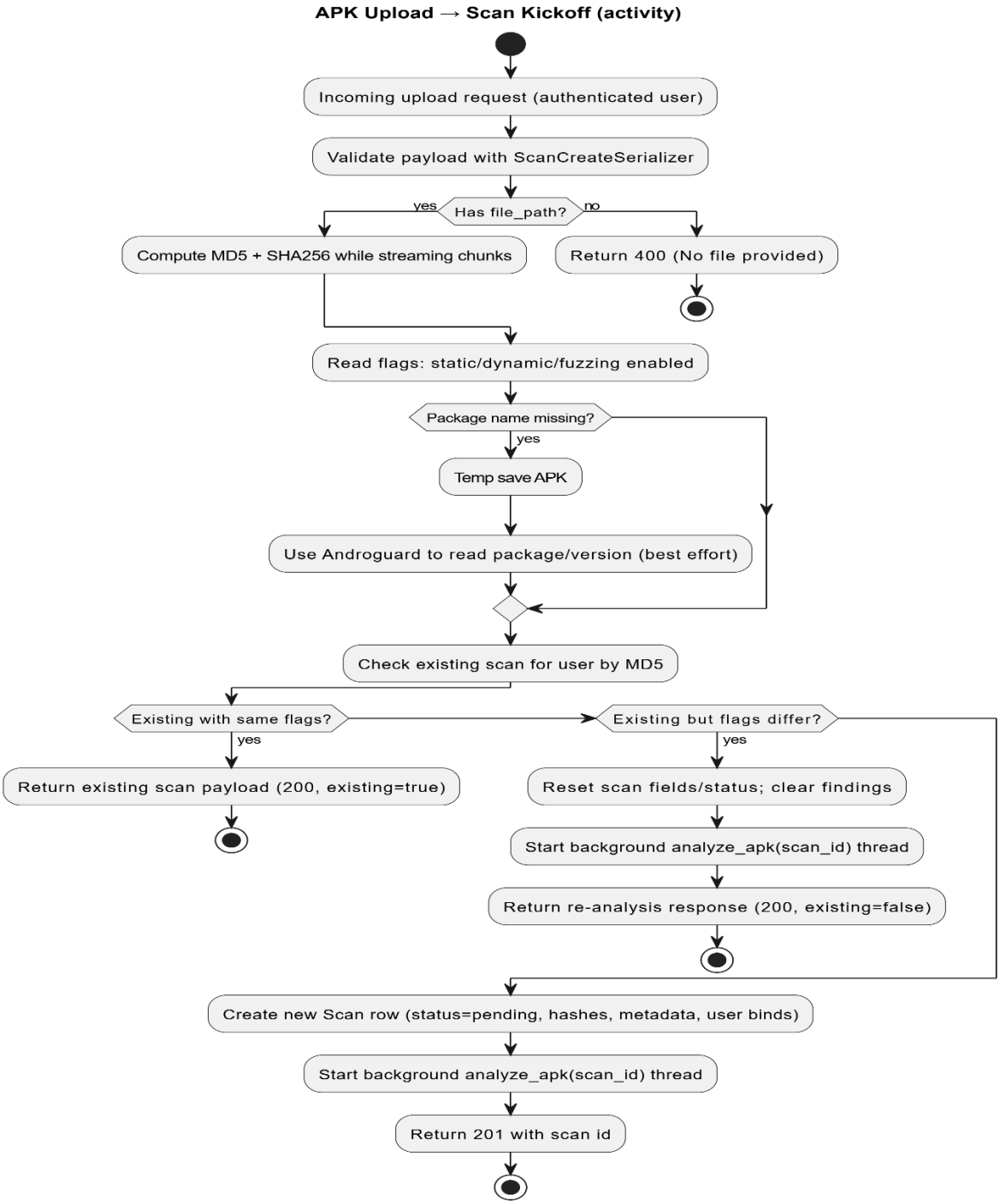


Figure 3.4: Activity Diagram 1 – Apk upload

2. Static Analysis

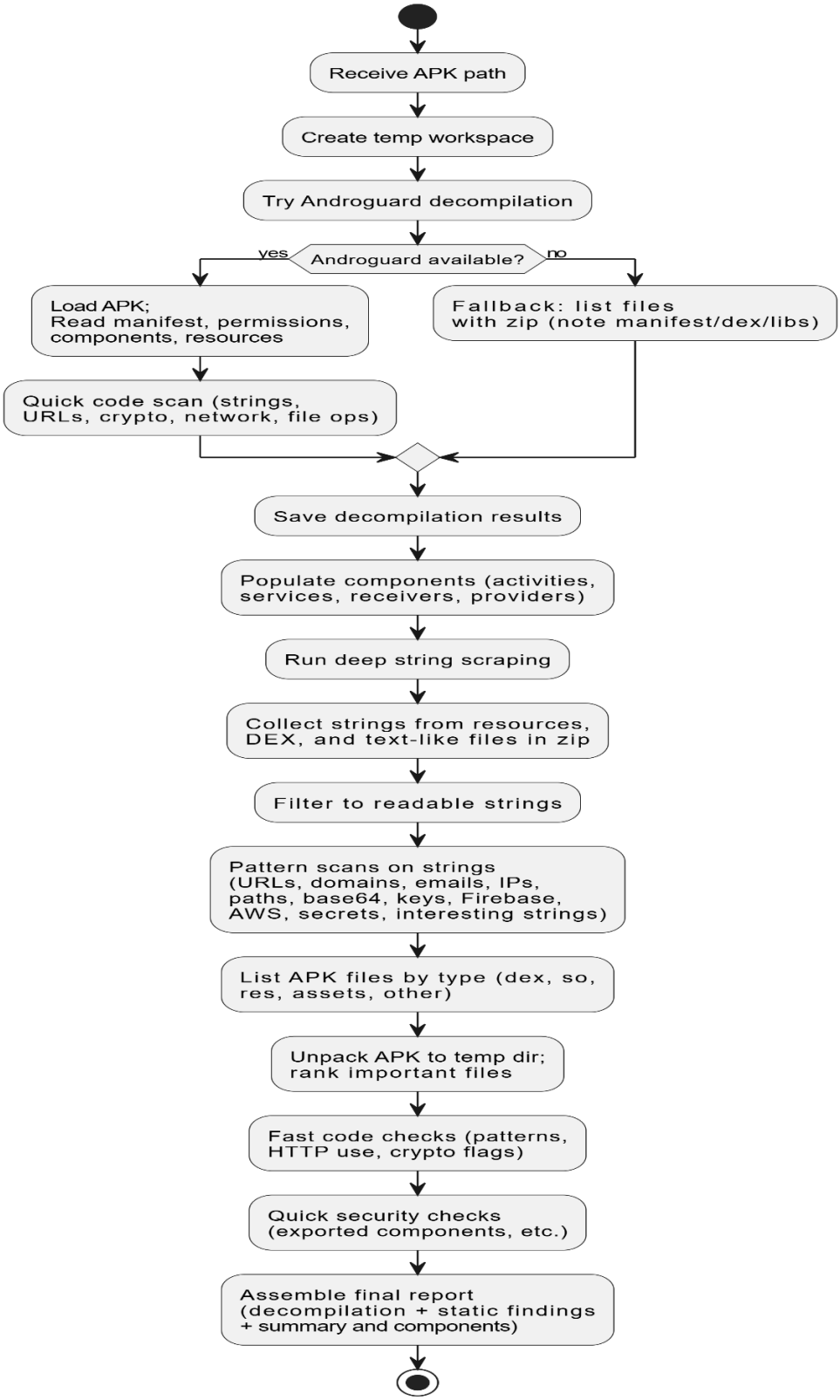


Figure 3.5: Activity Diagram 2 - Static Analysis

3. Dynamic Analysis

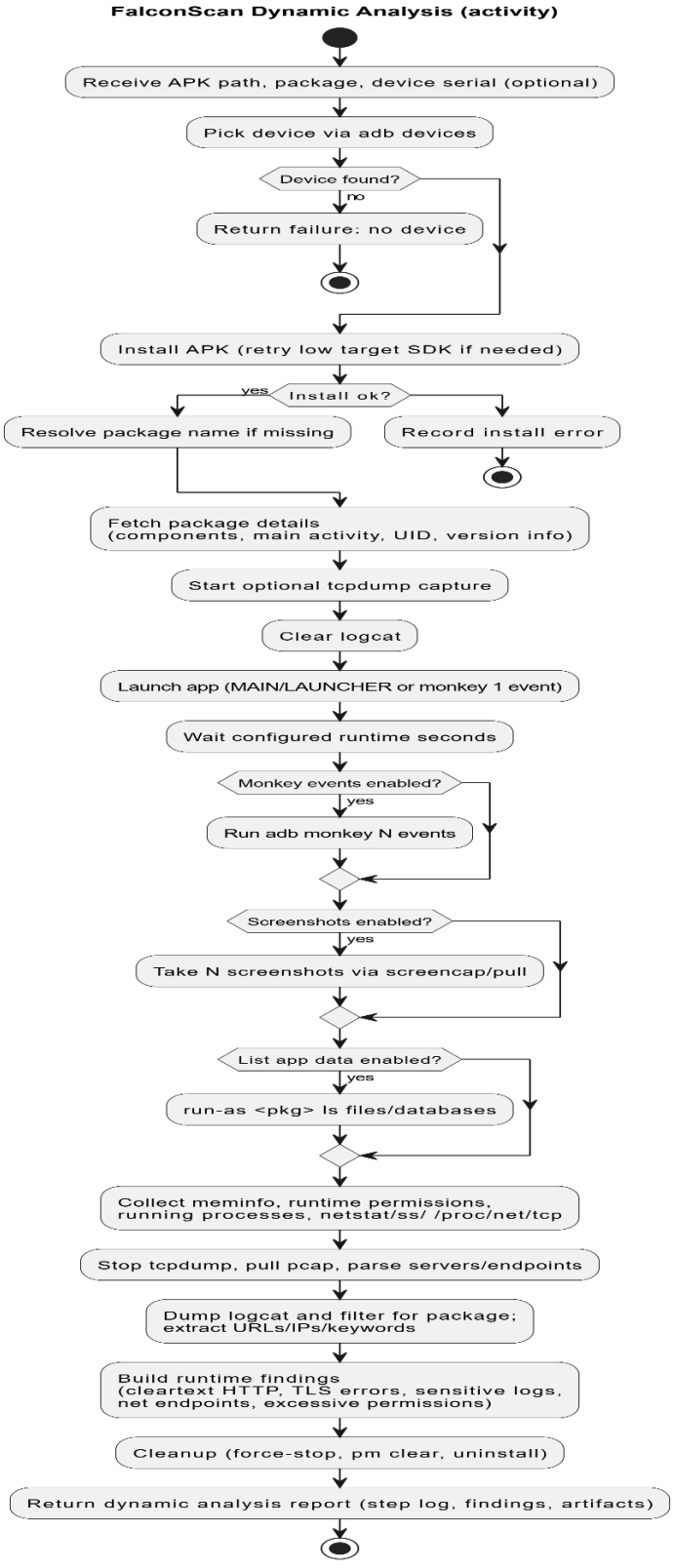


Figure 3.6: Activity Diagram 3 - Dynamic Analysis

4. AI/LLM

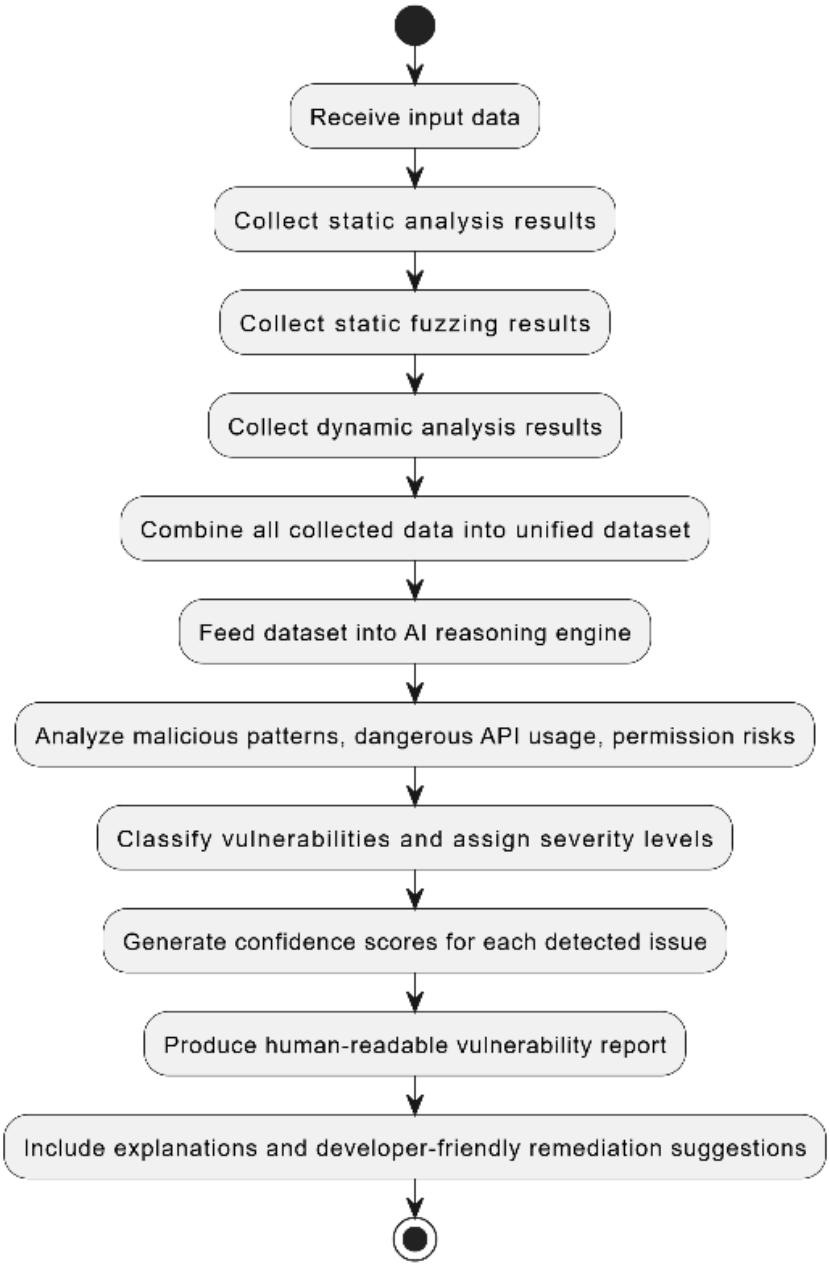


Figure 3.7: Activity Diagram 4 – AI/LLM

5. Reporting



Figure 3.8: Activity Diagram 5 – Report generation

6. Fuzzing

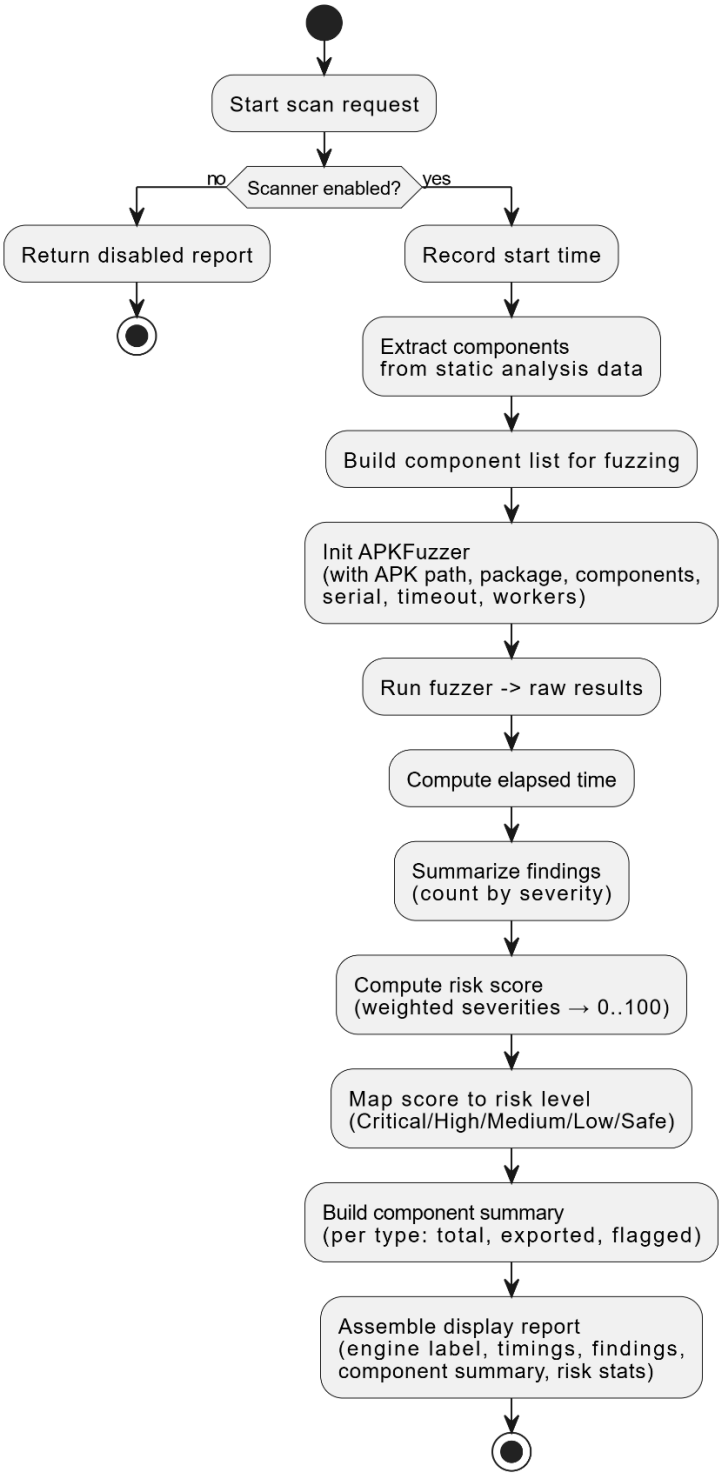


Figure 3.9: Activity Diagram 6 – Component fuzzing

3.4.1.2 Class Diagram

The class diagram provides a structural view of the system, highlighting the main classes, their attributes, methods, and the relationships among them. FalconScan's backend is primarily built using Django, and the frontend interacts with APIs via React/Next.js components.

Key Classes Included in the Diagram:

- **User** – Represents system users, including administrators and regular users. Key attributes include id, username, email, and is_staff. Core methods manage user state and permissions.
- **Scan** – Central model representing each APK scan. Attributes include app_name, file_path, threat_level, status, and hash_md5. Methods include __str__() for display purposes.
- **Vulnerability** – Associated with a specific scan, capturing information about detected vulnerabilities, such as vuln_type, description, severity, and location.
- **Permission** – Tracks granted permissions in APKs and links users to scans. Attributes include permission_type and granted_at.
- **Report** – Stores generated security reports, including report_path, generated_at, and status.
- **Serializers** – UserSerializer, RegisterSerializer, ScanSerializer, and ScanCreateSerializer handle validation and conversion between model instances and JSON responses for API communication.
- **ViewSets** – UserViewSet and ScanViewSet manage API endpoints for user and scan operations, encapsulating actions like upload, rescan, statistics, and download_pdf.
- **API Helpers** – authAPI and scansAPI manage frontend interactions with the backend, storing tokens and handling authentication flows, including JWT refresh handling.
- **Frontend Components** – LoginPage, UploadPage, and DashboardLayout consume API helpers and represent the interactive interface of the system.

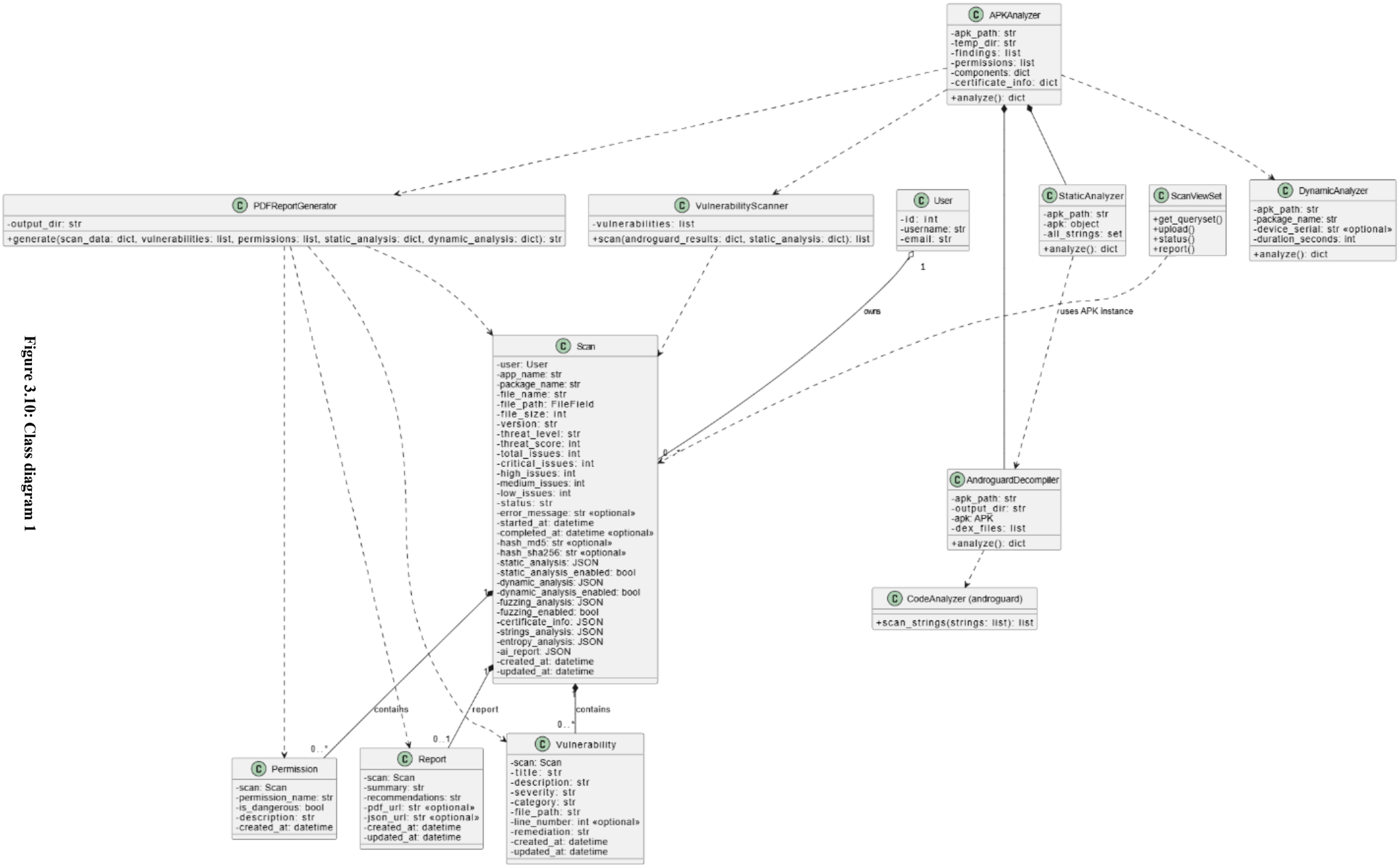


Figure 3.10: Class diagram 1

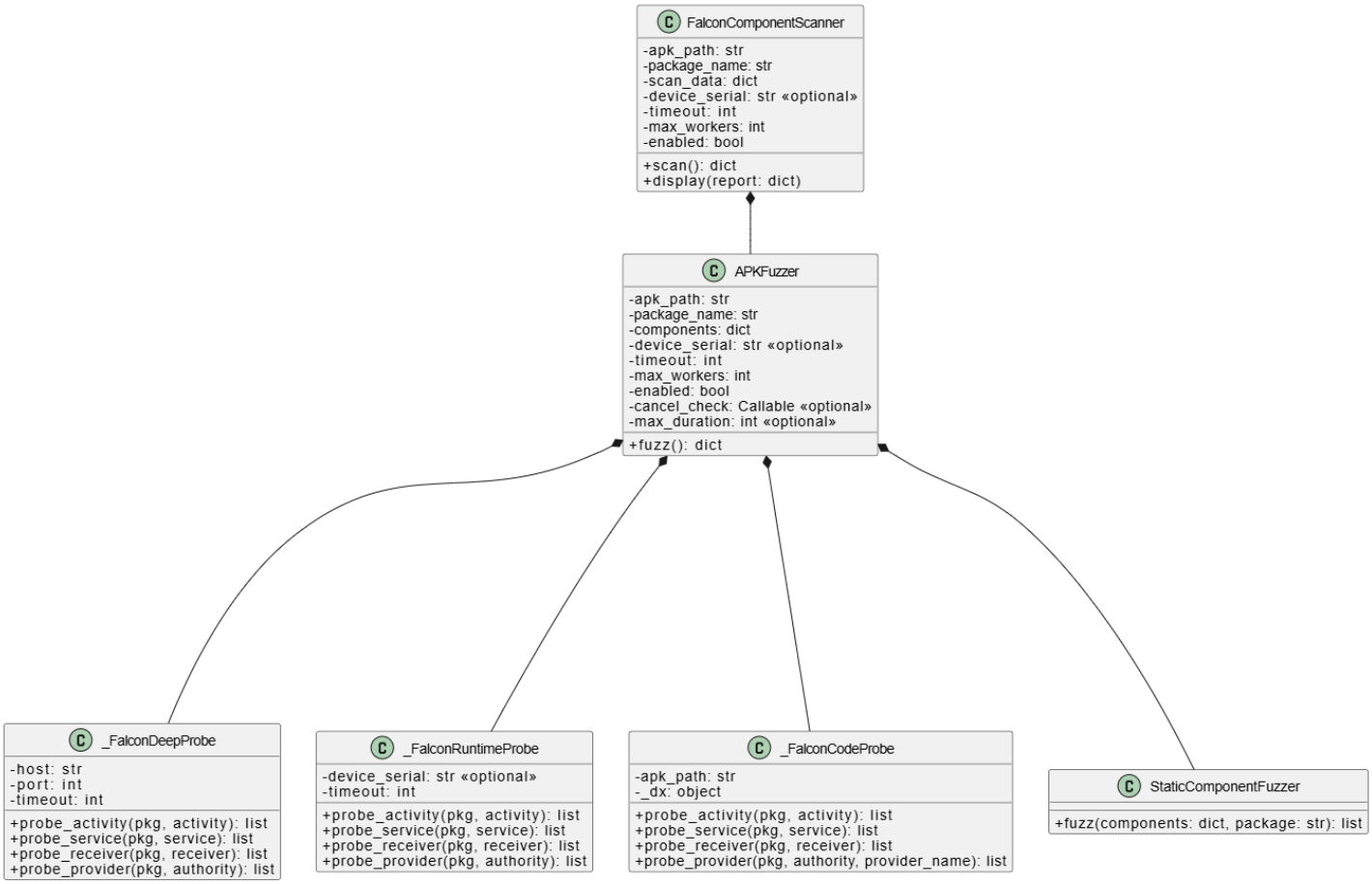


Figure 3.11: Class diagram 2

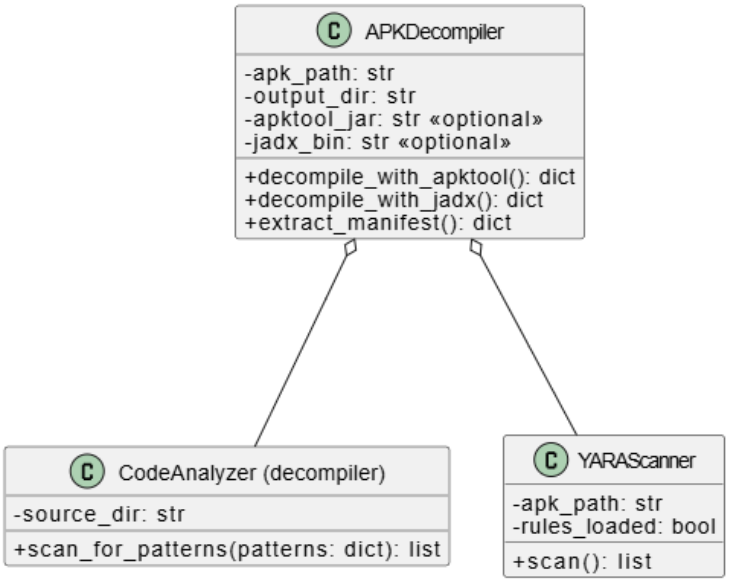


Figure 3.12: Class diagram 3

3.4.1.3 Sequence Diagram

1. User Authentication

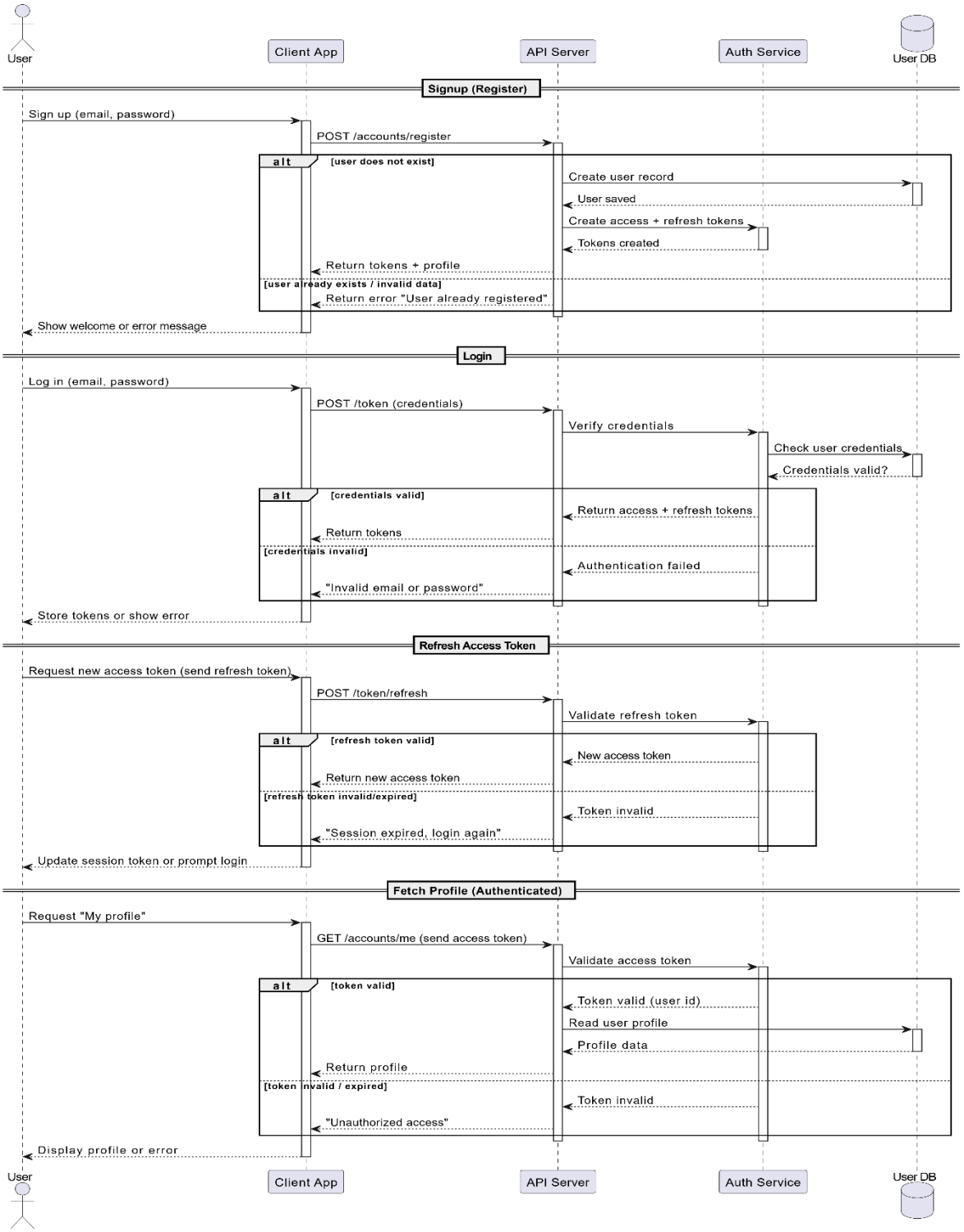


Figure 3.13: Sequence diagram 1 - User authentication

This sequence diagram illustrates the complete user authentication workflow of the system, including registration, login, token refresh, and profile retrieval.

In the signup process, the user submits their email and password through the client application. The client sends a registration request to the API server, which stores the new user record in the user database. After the user is successfully created, the authentication service generates an access token and a refresh token. These tokens are returned to the client application along with the user profile, allowing the user to start a session.

In the login process, the user enters their credentials in the client application. The application sends these credentials to the API server, which forwards them to the authentication service for verification. The authentication service checks the user credentials against the database. If the credentials are valid, it generates and returns a new access token and refresh token, which the client stores to maintain the user session.

When the access token expires, the refresh token is used to obtain a new access token. The client application sends the refresh token to the API server, which asks the authentication service to validate it. If the refresh token is valid, a new access token is issued and returned to the client, allowing the session to continue without requiring the user to log in again.

Finally, the authenticated user can request their profile information. The client application sends a request to the API server along with the access token. The API server verifies the token with the authentication service. Once validated, the server retrieves the user profile from the database and sends the profile data back to the client application, which then displays the information to the user.

2. APK Upload and Scan Initiation

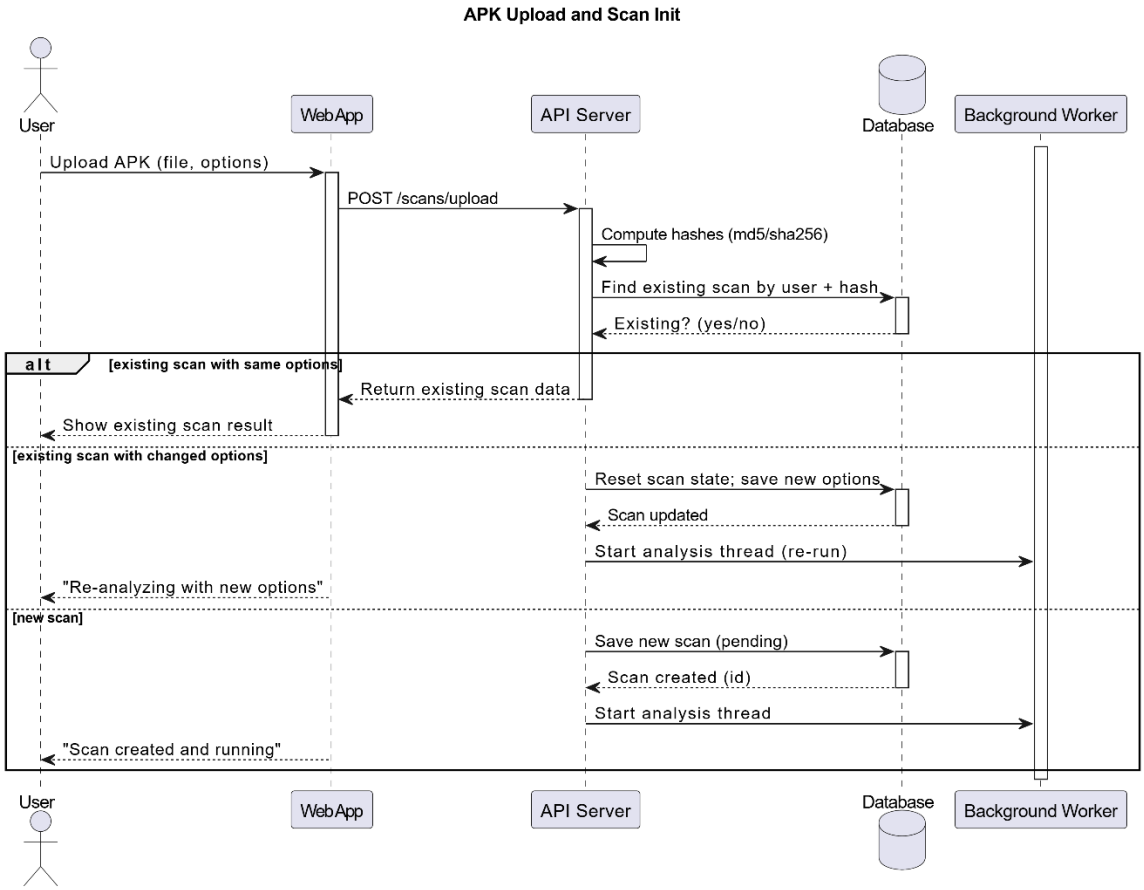


Figure 3.14: Sequence diagram 2- Apk upload

This sequence diagram illustrates the complete workflow for uploading an APK and starting a scan in the system.

When a user uploads an APK via the web application, the client sends the file along with selected options to the API server. The server first computes the file hashes (MD5 and SHA-256) to uniquely identify the APK and then queries the database to check if a scan for the same APK by the same user already exists.

There are three possible scenarios after this check:

1. **Existing scan with the same options** – The API returns the existing scan data directly to the web app, which displays the results to the user without re-running any analysis.

2. **Existing scan with changed options** – The API updates the existing scan record with the new options, resets its state, and triggers the background worker to re-run the analysis. The web app informs the user that the scan is being re-analyzed with the updated settings.
3. **New scan** – If no previous scan exists, the API creates a new scan record in the database, marks it as pending, and starts a background worker thread to perform the analysis. The web app notifies the user that the scan has been created and is running.

This design ensures efficient use of resources by avoiding duplicate scans while allowing users to re-run analyses when options change or when scanning a new APK. The use of alternate paths (alt / else) in the diagram represents these different possible outcomes in the workflow.

3. Static Analysis Flow

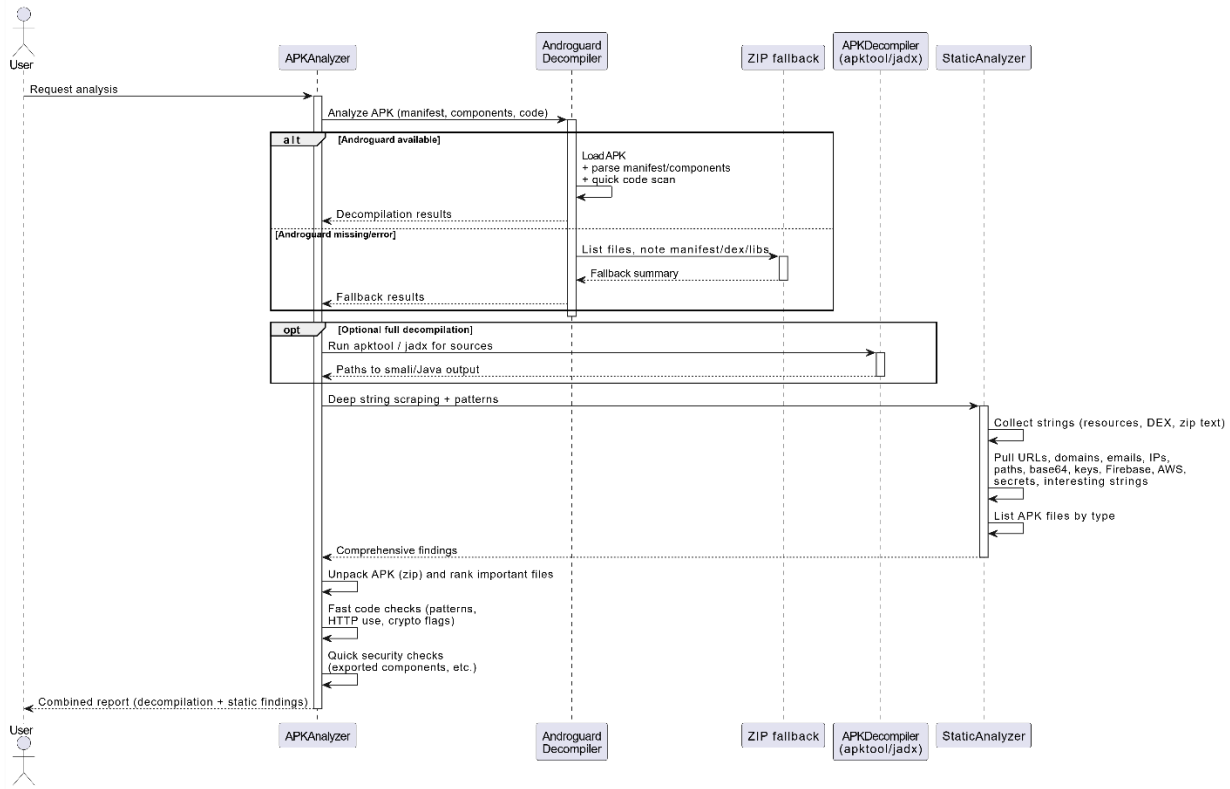


Figure 3.15: Sequence diagram 3 - Static analysis

This diagram represents the APK static analysis workflow with decompilation. When a user submits an APK to the APKAnalyzer (APA), it first attempts to use Androguard (AND) to extract the manifest, components, and perform a quick code scan. If Androguard is unavailable

or fails, a ZIP fallback extracts basic file information and returns it to APA. Optionally, APA can run external decompilers such as apktool or jadx to obtain smali or Java source files. APA then invokes the StaticAnalyzer (STA) to collect strings from resources, DEX files, and ZIP contents, while extracting URLs, domains, emails, IPs, paths, base64 blobs, cryptographic keys, Firebase/AWS configurations, secrets, and other interesting strings. The analyzer also lists all APK files by type and performs additional quick checks for HTTP calls, crypto usage, and exported components. Finally, APA aggregates all decompilation and static analysis results into a comprehensive report and returns it to the user, ensuring thorough analysis even if the primary decompiler fails.

4. Fuzzing Flow

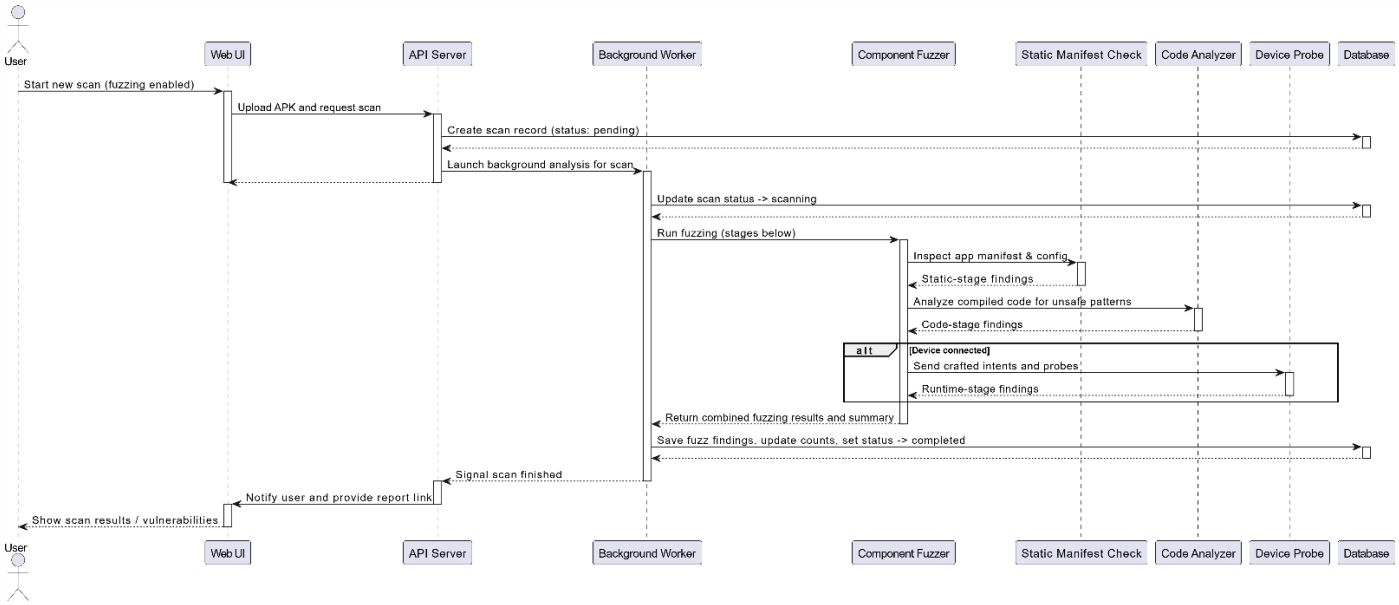


Figure 3.16: Sequence diagram 4 - Fuzzing

After the user starts a scan through the Web UI, the APK is uploaded to the API Server. The API creates a new scan record in the Database with status *pending* and sends the job to the Background Worker. The worker updates the scan status to *scanning* and starts the Component Fuzzer, which controls the analysis stages.

The Component Fuzzer first calls the Static Manifest Check module. This stage reads the Android manifest and configuration files to identify permissions, exported components, and

other settings that affect component exposure. Any risky configurations found are returned as static stage findings.

Next, the fuzzer invokes the Code Analyzer to inspect the application code for unsafe patterns. This module searches for issues such as insecure intent handling, risky API usage, or suspicious code behavior. The results are returned as code stage findings.

If a device or emulator is connected, the system performs runtime fuzzing. The fuzzer sends crafted intents and probes to the app through the Device Probe to interact with Activities, Services, Broadcast Receivers, and Content Providers. The device returns runtime observations such as crashes or abnormal responses, which are recorded as runtime findings.

After all stages finish, the Component Fuzzer combines the findings and sends the results to the Background Worker. The worker stores the results in the Database, updates the scan status to *completed*, and notifies the API Server. The API then informs the Web UI, which displays the final vulnerability report to the User.

5. Dynamic Analysis Flow

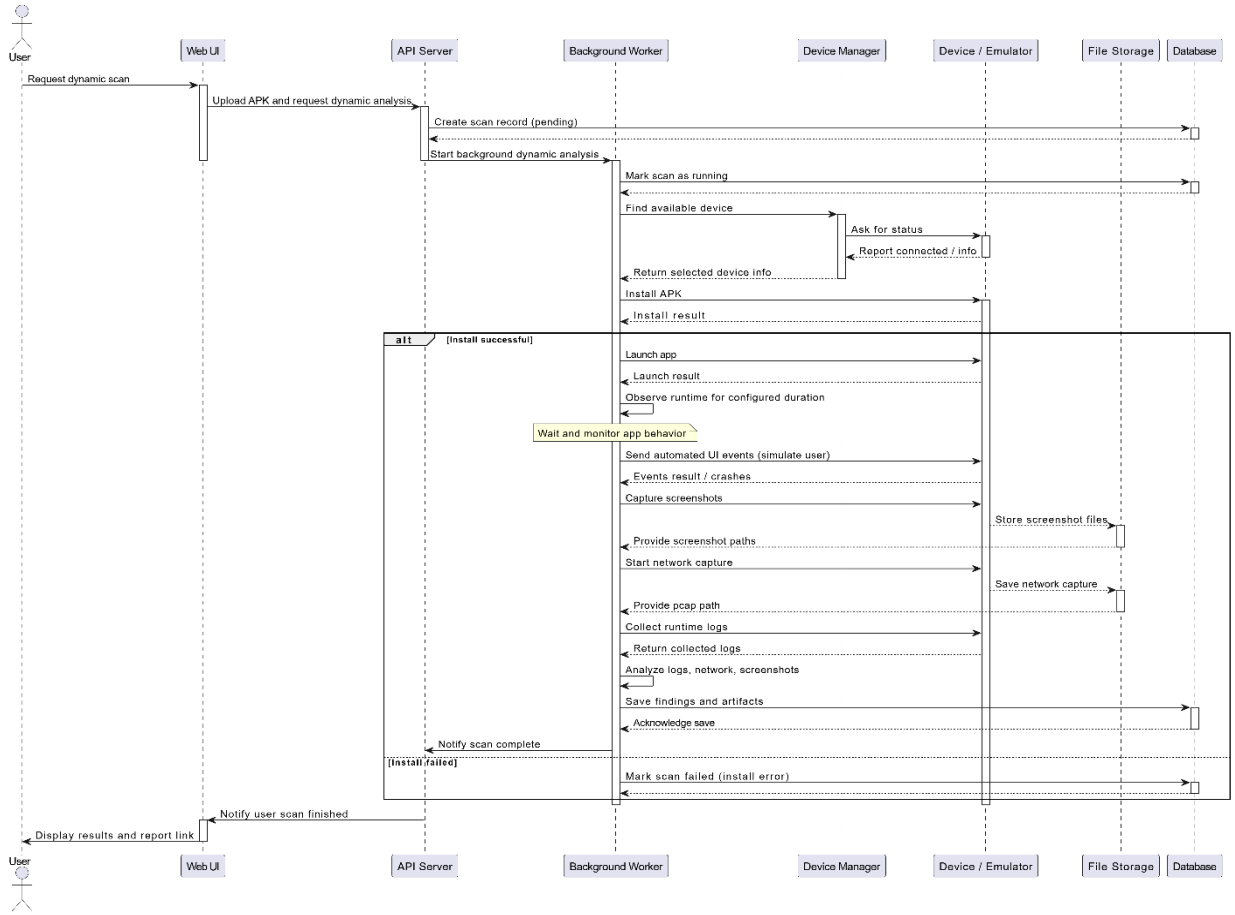


Figure 3.17: Sequence diagram 5 – Dynamic Analysis

The User requests a dynamic scan through the Web UI, which uploads the APK to the API Server. The API creates a scan record in the Database and forwards the job to the Background Worker. The worker updates the scan status to running and requests an available device from the Device Manager. Once a device is selected, the worker installs the APK and checks for installation success.

If the install fails, the worker marks the scan as failed. If successful, the worker launches the app, observes its runtime behavior, sends automated UI events, captures screenshots, network traffic, and runtime logs. The worker analyzes all collected data and saves the findings in the database. Finally, the API notifies the Web UI, which presents the scan results and report to the user.

6. AI/LLM Reasoning and Correlation Flow

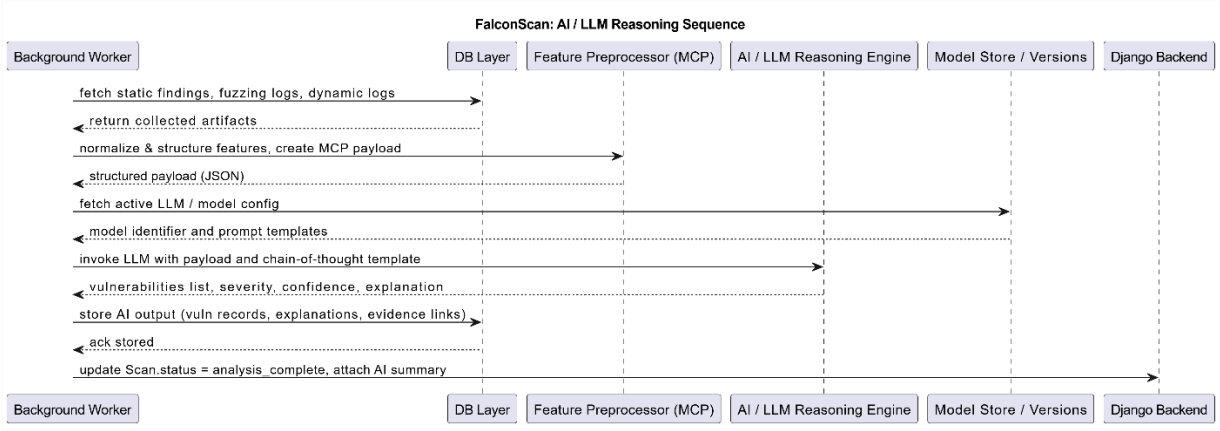


Figure 3.18: Sequence diagram 6 - AI/LLM reasoning

This diagram explains how FalconScan merges diverse evidence and uses an LLM-based reasoning engine to produce coherent vulnerability classifications and explanations. The worker collects static findings, fuzzing artifacts, and dynamic logs from the database, then passes the raw information to a feature preprocessor implementing the Model Context Protocol. The preprocessor normalizes fields, extracts salient tokens, and formats the payload in a consistent structure so the LLM receives deterministic inputs. The worker selects the active model and prompt templates from a model store, which supports versioning and rollback. The LLM is invoked with the structured payload and guided prompt templates that request classification, severity scoring, confidence estimation, and an explanatory chain of reasoning. The AI output is then persisted into the DB as vulnerability records and supporting evidence links. This approach makes LLM outputs auditable and traceable because the payload and prompt templates are archived. Given that LLMs may hallucinate, the system logs confidence scores and references to original artifacts so a human analyst can verify the reasoning. Model governance items such as rate limits, prompt guardrails, and offline fine tuning hooks are managed by the model store.

7. Report Generation and Storage

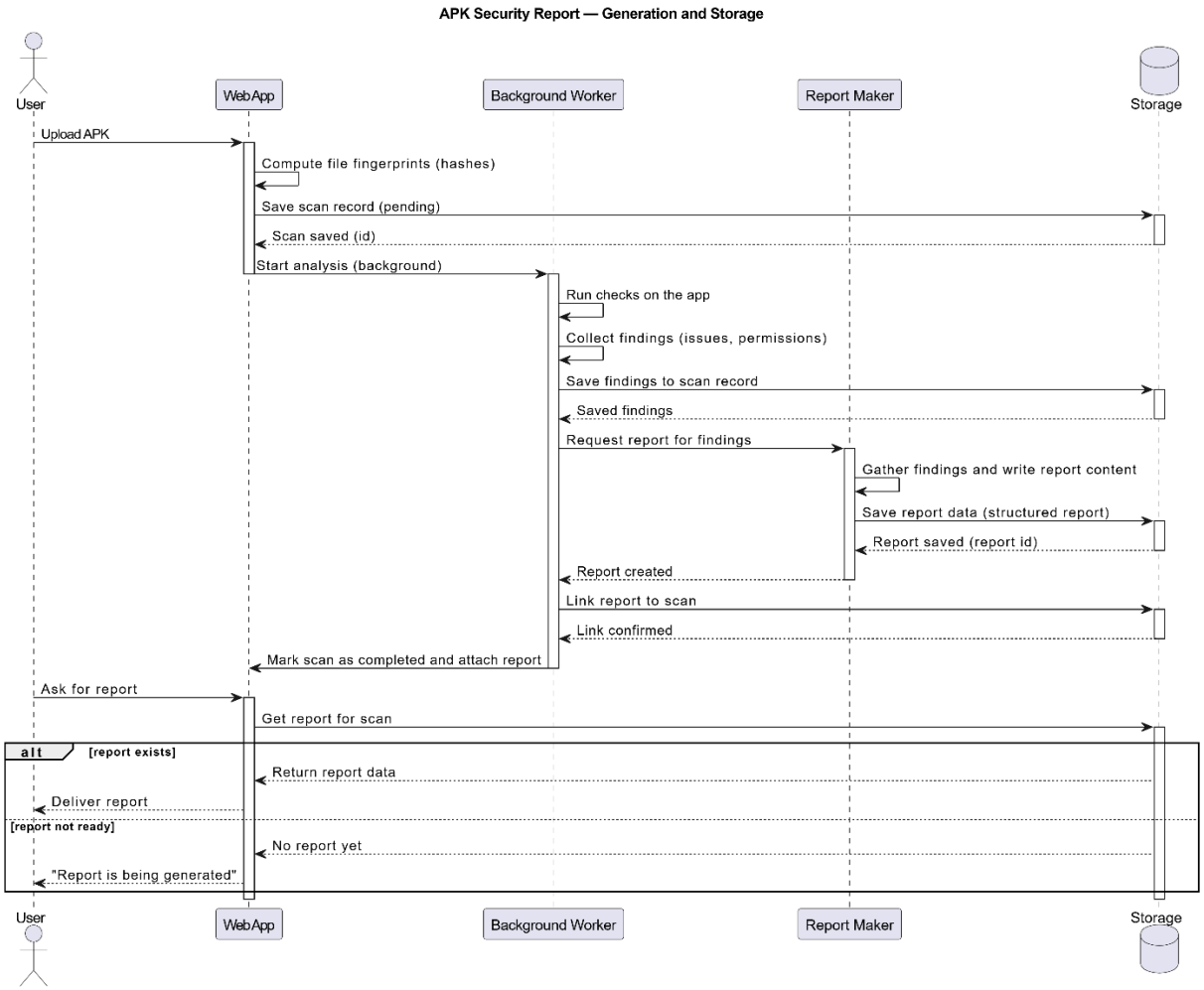


Figure 3.19: Sequence diagram 7 - Report generation

The sequence diagram shows how an APK security report is generated and delivered to the user. First, the user uploads an APK through the Web App. The application calculates file hashes and stores a new scan record in the storage database. After saving the scan, the Web App starts a background worker to perform the analysis.

The worker runs various security checks on the application and collects findings such as vulnerabilities and permissions. These results are then saved to the database. Once the analysis is complete, the worker requests the report maker to generate a structured report based on the

collected findings. The generated report is stored in the database and linked to the related scan record.

Finally, when the user requests the report, the Web App retrieves it from storage. If the report is available, it is returned to the user. If it has not yet been generated, the user is informed that the report is still being prepared.

8. Dashboard Fetch, Download PDF, Feedback and Rescan

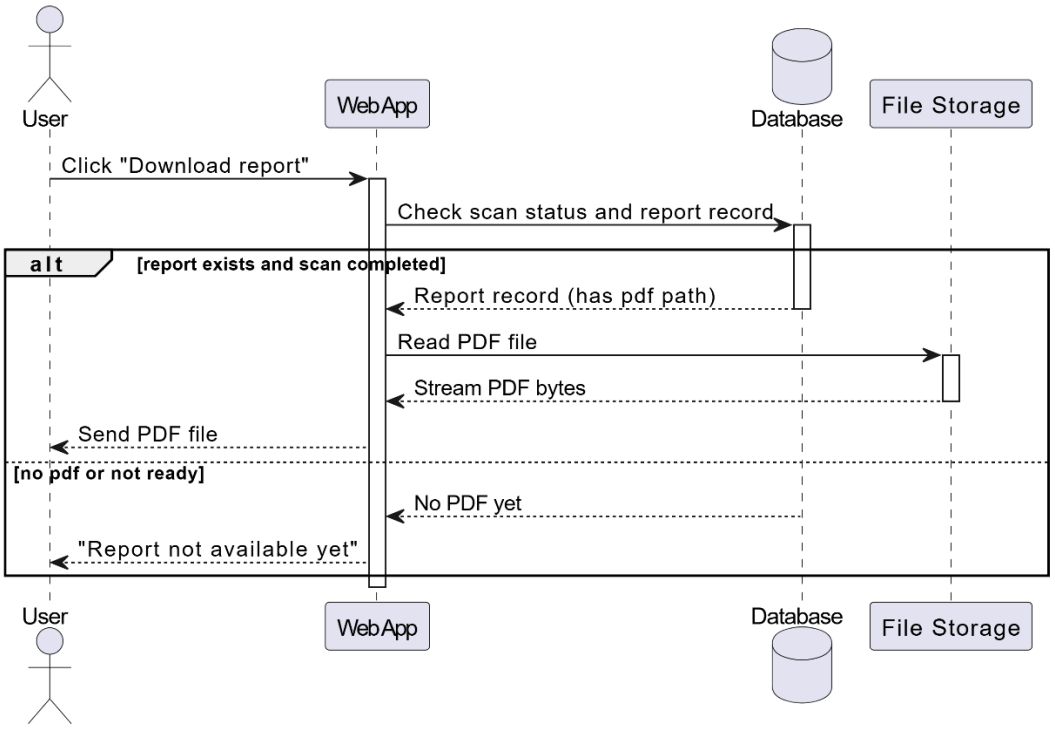


Figure 3.20: Sequence diagram 8 – Download PDF

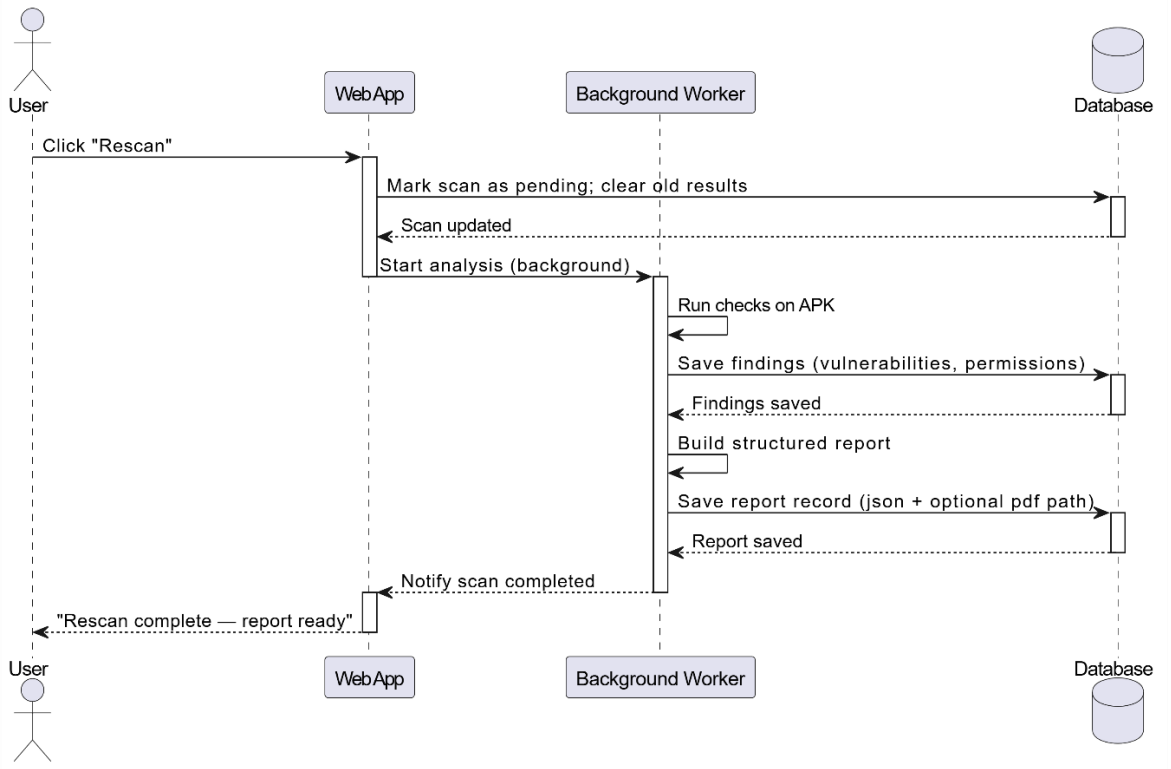


Figure 3.21: Sequence diagram 9 - Rescan and report recreation

3.5 Data Design

This section describes how the system's information domain is converted into structured data models and storage schemas.

FalconScan uses a relational database model suitable for handling structured APK metadata and vulnerability datasets.

3.5.1 Major Entity Storage

- **auth_user Table**: Stores core user authentication details such as username, email, hashed password, and access flags (staff, superuser, active). It also keeps audit timestamps like date joined and last login.

- **user_profiles Table**: Stores extended profile information for each user including avatar, biography, notification preferences, two-factor authentication status, and optional API key for integrations.
- **scans Table**: Stores metadata and results for each uploaded APK scan including file details, hashes, threat score, scan status, timestamps, and multiple analysis outputs such as static, dynamic, fuzzing, certificate, strings, and entropy analysis.
- **vulnerabilities Table**: Stores security issues discovered during analysis, including title, description, severity level, category, file location, and remediation guidance linked to the scan.
- **permissions Table**: Stores Android permissions extracted from the APK, identifying whether each permission is dangerous and providing descriptive context.
- **reports Table**: Stores generated security reports for each scan including summary, recommendations, and links to downloadable PDF or JSON reports.

The Django ORM structures these models and enforces relationships through foreign keys and constraints, ensuring referential integrity between users, scans, vulnerabilities, permissions, and reports.

3.5.2 Data Dictionary

Table 3.2: Key Fields in the FalconScan Scan Results Database

Table	Field	Type	Description
user_profiles	user_id	Integer (FK: auth_user.id)	Owner account for this profile
	avatar	Image	Optional user avatar stored under avatars/
	bio	Text	Freeform user bio
	two_factor_enabled	Boolean	Whether 2FA

Table	Field	Type	Description
			is enabled
	notifications_enabled	Boolean	Receive general notifications
	threat_alerts	Boolean	Receive threat/scan alerts
	report_delivery	Boolean	Receive report delivery notices
	api_key	String (max 100)	Optional unique API key for integrations
	created_at	DateTime	When the profile was created
	updated_at	DateTime	Last profile update
scans	id	Integer (PK)	Unique ID for each scan record
	user_id	Integer (FK: auth_user.id)	Owner of the scan
	app_name	String (255)	Human-readable app name
	package_name	String (255)	Android

Table	Field	Type	Description
			package ID
	file_name	String (255)	Uploaded APK file name
	file_path	File	Stored APK path under media/apks/
	file_size	BigInteger	APK size in bytes
	version	String (50)	App version name (if available)
	threat_level	Enum (critical/high/medium/low/safe)	Overall risk rating
	threat_score	Integer	Numeric risk score
	total_issues	Integer	Count of all detected issues
	critical_issues	Integer	Count of critical issues
	high_issues	Integer	Count of high issues
	medium_issues	Integer	Count of medium issues
	low_issues	Integer	Count of low issues
	status	Enum	Scan

Table	Field	Type	Description
		(pending/scanning/completed/failed)	workflow state
	error_message	Text	Error details if scan failed
	started_at	DateTime	When the scan began
	completed_at	DateTime (nullable)	When the scan finished
	hash_md5	String (32)	MD5 checksum of the APK
	hash_sha256	String (64)	SHA-256 checksum of the APK
	static_analysis	JSON	Parsed static analysis results
	static_analysis_enabled	Boolean	Whether static analysis ran
	dynamic_analysis	JSON	Parsed dynamic analysis results
	dynamic_analysis_enabled	Boolean	Whether dynamic analysis ran
	fuzzing_analysis	JSON	Parsed fuzzing

Table	Field	Type	Description
			results
	fuzzing_enabled	Boolean	Whether fuzzing ran
	certificate_info	JSON	Signing certificate details
	strings_analysis	JSON	Extracted string analysis results
	entropy_analysis	JSON	Entropy / packing indicators
	ai_report	JSON	Cached AI-generated security report
	created_at	DateTime	Record creation timestamp
	updated_at	DateTime	Last update timestamp
	Constraint	unique_user_apk	(user_id, hash_md5) must be unique
vulnerabilities	id	Integer (PK)	Unique vulnerability record
	scan_id	Integer (FK: scans.id)	Scan this

Table	Field	Type	Description
			finding belongs to
	title	String (255)	Vulnerability title
	description	Text	Details of the issue
	severity	Enum (critical/high/medium/low/info)	Severity level
	category	String (100)	Category/type of issue
	file_path	String (500)	File path within APK (if applicable)
	line_number	Integer (nullable)	Line number (if applicable)
	remediation	Text	Recommended fix
	created_at	DateTime	When recorded
	updated_at	DateTime	Last update
permissions	id	Integer (PK)	Unique permission record
	scan_id	Integer (FK: scans.id)	Related scan
	permission_name	String (255)	Android permission name
	is_dangerous	Boolean	Whether marked

Table	Field	Type	Description
			dangerous
	description	Text	Optional extra context
	created_at	DateTime	When recorded
reports	id	Integer (PK)	Unique report record
	scan_id	Integer (FK: scans.id, 1–1)	Scan this report summarizes
	summary	Text	Narrative summary of findings
	recommendations	Text	Recommended mitigations
	pdf_url	String (1000, nullable)	Path/URL to generated PDF
	json_url	Text (nullable)	Path/URL to JSON export
	created_at	DateTime	When report was created
	updated_at	DateTime	Last update
auth_user	id	Integer (PK)	User ID
	username	String	Login name (unique)
	email	String	Email address
	password	String	Hashed password

Table	Field	Type	Description
	first_name / last_name	String	Names
	is_staff / is_superuser / is_active	Boolean	Access flags
	date_joined / last_login	DateTime	Audit fields

4 Chapter 4: Project Methodology & Algorithms

The implementation of FalconScan, an advanced Android APK vulnerability scanner, was carried out in a systematic and structured manner to ensure that every component of the system works efficiently, securely, and intelligently. The project methodology focuses on transforming raw APK files into actionable vulnerability reports using static analysis, dynamic analysis, fuzzing, and an AI reasoning engine. The methodology also emphasizes adherence to coding standards, modular development, and secure handling of sensitive data. The system's core algorithms are designed to enforce business rules, perform optimization, and implement intelligent features such as vulnerability classification and risk ranking.

4.1.1 Project Methodology (Step-by-Step Approach)

The methodology adopted for FalconScan consists of multiple stages. Each stage is critical for achieving accurate vulnerability detection, precise risk evaluation, and actionable recommendations. These stages are described in detail below.

1. Data Collection

The first stage of the implementation involves collecting relevant data from the APK files. This data is critical for both static and dynamic analysis and for feeding the AI reasoning engine.

The collected data includes static code features extracted from decompiled APKs, manifest configurations, API usage patterns, dynamic behavior logs obtained during runtime execution, and crash logs from fuzzing. Each dataset provides insight into the potential vulnerabilities, misuse of permissions, network activity, and runtime anomalies.

Tools such as APKTool and JADX were used to decompile APKs, extract AndroidManifest.xml files, resources, assets, and smali code. For dynamic data, APKs were executed on an isolated Android Virtual Device (AVD) with full monitoring enabled. Runtime events such as system calls, network traffic, file system access, API calls, and crash events were collected using strace, ltrace, logcat, and network proxy logging.

Data collection is essential to generate structured input for the AI reasoning engine. Static analysis provides insight into potential vulnerabilities and misconfigurations, dynamic analysis reveals runtime behavior, and fuzzing uncovers crash points and robustness issues. Together, these datasets allow FalconScan to provide a holistic assessment of APK security.

2. Data Preprocessing

After collecting raw data, preprocessing ensures the inputs are clean, consistent, and formatted correctly for analysis and AI reasoning.

Some dynamic monitoring logs may contain missing entries due to process crashes or incomplete network events. These missing values are handled using forward filling or interpolation techniques to maintain consistency in temporal datasets.

Duplicate log entries or repeated API calls are removed using unique identifiers and timestamp-based filtering. This ensures that the AI model does not overweight repeated actions and prevents skewing the classification results.

Numerical features such as API call frequencies, packet sizes, and crash counts are normalized using Min-Max scaling to bring all features into a consistent range. This ensures that no single feature dominates the AI model's reasoning.

Preprocessing also includes organizing data into structured JSON-like records, making it easier for the AI pipeline to parse and analyze vulnerabilities.

3. Feature Extraction & Selection

Feature extraction is a critical step to reduce noise and retain only relevant information for vulnerability detection.

The key features extracted include:

1. Permissions requested in AndroidManifest.xml

2. Exported activities, services, and receivers
3. API usage patterns such as network, cryptography, and system commands
4. Runtime behavior patterns including file system access, network endpoints, and dynamic code loading
5. Fuzzing crash signatures and anomalies

Static features are extracted using regex-based parsers and AST (Abstract Syntax Tree) analysis on decompiled code. Dynamic features are collected using structured logs from the emulator environment. Redundant features or highly correlated ones are removed using correlation analysis and heuristic filtering. Only features that directly impact security risk assessment are retained for AI analysis.

Feature selection ensures that the AI reasoning engine focuses on actionable patterns and reduces computational overhead.

4. Model Training and AI-Based Analysis

The AI layer is the core innovation of FalconScan. It transforms raw analysis data into human-readable vulnerabilities and risk rankings using a reasoning engine based on Large Language Models (LLMs).

The AI reasoning engine is designed to classify vulnerabilities, determine severity levels, and provide remediation guidance.

FalconScan does not rely on external datasets. Instead, the AI reasoning engine operates on structured features obtained from each APK. The pipeline ensures consistent formatting and context structuring using a Model Context Protocol (MCP).

Using an AI reasoning engine allows FalconScan to generate intelligent and contextual explanations for vulnerabilities rather than just listing them. The system can detect complex patterns such as dangerous API combinations, behavioral anomalies, and permission-risk relationships that traditional static or dynamic tools might miss.

5. Advanced Techniques

NLP and AI Analysis: The AI reasoning engine uses natural language processing internally to generate human-readable vulnerability reports. Features from static and dynamic analysis are transformed into structured prompts, and the model generates explanations, risk categorization, severity scores, and remediation suggestions.

Dynamic Analysis Integration: Runtime behavior logs from system calls, network traffic, file system monitoring, and API hooking are fed into the AI engine. This allows FalconScan to reason about behavioral anomalies in addition to code-based vulnerabilities.

Fuzzing: Automated fuzzing is applied to identify crash points, boundary conditions, and unexpected input handling issues. Fuzzing results become additional features for the AI reasoning engine, enriching the model's understanding of potential APK vulnerabilities.

6. Deployment

The FalconScan system is deployed on a local on-premise environment. The frontend is built using Next.js and runs on port 3000, while the backend is a Django REST API server managing APK uploads, scan requests, and report generation.

The final system provides a web-based dashboard for administrators, developers, and security analysts to upload APKs, monitor scans, and view detailed vulnerability reports.

Deployment on a Windows machine ensures controlled execution of dynamic analysis and isolated emulation. The system can handle multiple concurrent scans while keeping runtime monitoring secure and reproducible.

Real-time access to scan results allows developers and security analysts to quickly understand and remediate security issues in APKs, improving the overall security posture of applications.

4.1.2 Algorithm

The core strength of FalconScan lies in its ability to intelligently analyze APKs, detect vulnerabilities, and generate actionable reports. The algorithms below avoid trivial CRUD operations and focus on features that enforce business rules, perform optimization, and implement AI-based reasoning.

Algorithm 1: Vulnerability Classification (AI-Based)

Input: Structured APK Analysis Data (static features, dynamic logs, fuzzing outputs)
Output: Classified Vulnerabilities with Severity and Remediation Suggestions

Pseudocode:

1: procedure VulnerabilityClassification(APKData)
2: PreprocessedData ← preprocess(APKData)
3: FeatureVector ← extract_features(PreprocessedData)
4: ContextualInput ← apply_ModelContextProtocol(FeatureVector)
5: ClassificationResult ← LLM.predict(ContextualInput)
6: for each Vulnerability in ClassificationResult do
7: assign SeverityScore based on CVSS mapping
8: generate HumanReadableExplanation(Vulnerability)
9: suggest RemediationSteps(Vulnerability)
10: end for
11: return ClassificationResult
12: end procedure

Explanation:

This algorithm is the central intelligence of FalconScan. It takes structured input from static analysis, dynamic logs, and fuzzing, preprocesses it, and then feeds it into the AI reasoning engine. The model predicts vulnerability categories, severity levels, and generates explanations and remediation guidance. This ensures the output is actionable and developer-friendly.

Algorithm 2: Static Analysis & Feature Extraction

Input: APK File (path)

Output: Structured Feature Set (Permissions, Exported Components, API Patterns)

Pseudocode:

1: procedure StaticAnalysis(APKFile)

2: DecompiledFiles ← decompile(APKFile) using APKTool and JADX

3: ManifestData ← extract_manifest(DecompiledFiles)

4: Permissions ← analyze_permissions(ManifestData)

5: ExportedComponents ← check_exported_components(ManifestData)

6: CodePatterns ← scan_code_patterns(DecompiledFiles) using AST and Regex

7: RiskFlags ← identify_insecure_features(Permissions, ExportedComponents, CodePatterns)

8: FeatureSet ← combine(Permissions, ExportedComponents, CodePatterns, RiskFlags)

9: return FeatureSet

10: end procedure

Explanation:

This algorithm handles static analysis of APK files. It extracts permissions, exported activities, and code patterns. The analysis detects dangerous configurations, insecure API usage, and hardcoded secrets. By generating a structured feature set, it provides the foundation for AI reasoning and further dynamic analysis.

Algorithm 3: Dynamic Behavior Monitoring

Input: APK File, Emulator Configuration

Output: Runtime Behavior Logs (Network, System Calls, File Activity, API Hooks)

Pseudocode:

1: procedure DynamicAnalysis(APKFile, Emulator)

2: Launch Emulator with NetworkProxy, FileMonitor

3: Install APKFile using adb install

4: StartMainActivity(APKFile)

5: TriggerAutomatedEvents(Clicks, Swipes, Inputs)

6: MonitorSystemCalls ← capture_strace_ltrace()

7: MonitorNetwork ← capture_traffic(HTTP, HTTPS, DNS)

8: MonitorFiles ← log_file_access(FileSystem)

9: CollectCrashEvents ← monitor_logcat()

10: RuntimeLogs ← aggregate(MonitorSystemCalls, MonitorNetwork, MonitorFiles, MonitorAPIs, CollectCrashEvents)

11: return RuntimeLogs

12: end procedure

Explanation:

This algorithm conducts dynamic analysis by executing the APK in a controlled emulator. It logs system calls, network traffic, file access, API usage, and crash events. This behavior data helps identify anomalies that static analysis cannot detect, providing a richer dataset for AI-based classification and risk scoring.

These three algorithms together form the backbone of FalconScan. Static analysis ensures thorough feature extraction, dynamic analysis adds behavioral context, and the AI reasoning engine intelligently classifies vulnerabilities with actionable guidance.

4.2 Training Results and Model Evaluation

Although FalconScan does not ship a locally trained LLM, it uses structured features produced by static analysis, dynamic instrumentation, and fuzzing to drive an LLM-based reasoning engine (the project's `ai_analyzer` constructs a compact prompt and calls a model). The evaluation focus is therefore on inference quality, calibration, and human-in-the-loop feedback rather than on an internal end-to-end model training pipeline.

1. Dataset Overview

FalconScan generates its inputs internally. The AI reasoning engine consumes a curated, structured representation derived from:

- Static analysis outputs: manifest-derived fields (exported components, permissions, intent-filters) and summarized code findings.
- Dynamic analysis logs: time-stamped traces such as logcat excerpts, exception traces, system-call summaries, network captures metadata, and instrumentation hooks.
- Fuzzing outputs: crash reports, exception summaries, payloads that triggered abnormal behaviour, and provenance metadata.

These artifacts are flattened and truncated into token-budget-aware sections by 'ai_analyzer._build_prompt()' and combined into a consistent feature-like JSON structure for the LLM. To enable later supervised evaluation or retraining, FalconScan should persist paired examples of (prompt, LLM response, ground-truth label) and provenance metadata.

2. Training Procedure

- Platform & Role: FalconScan currently performs inference against external LLMs. There is no end-to-end in-repo model training step; instead the project supports a model-cascade and fallback logic for inference.
- Batch processing & prompt pipelines: Inputs from multiple scans are normalised and formatted consistently before being sent to the LLM; prompt sections are length-limited and capped to stay within token budgets.
- Feature normalisation: Numerical summaries, categorical tags, and truncated evidence snippets are standardised (e.g., severity labels, timestamps, evidence snippets) to ensure consistent LLM prompts and downstream metric computation.
- Model Context Protocol: FalconScan enforces a predictable prompt schema (exact JSON keys requested in the prompt) so the LLM returns structured, machine-parseable output. The prompt builder trims and sanitises content to reduce hallucination risk.

- (If/When retraining): When a retraining workflow is adopted, use curated labelled examples collected from human triage (see below) and maintain dataset versioning and validation splits. For now, correctness is assessed by post-hoc comparison of AI outputs to known evidence and human labels rather than internal gradient-based training.

3. Evaluation Metrics

Because FalconScan's AI layer is inference-first, evaluation is performed by comparing LLM outputs against ground-truth labels created by human reviewers or automated heuristics. Recommended metrics:

- Accuracy: fraction of correctly classified findings (where a ground-truth label exists).
- Precision: proportion of reported vulnerabilities that are true positives (helps reduce analyst overhead).
- Recall: fraction of real vulnerabilities the AI identified (coverage).
- Calibration / Confidence: measure whether the LLM's confidence or assigned confidence score matches empirical correctness (e.g., expected vs observed correctness in confidence buckets).
- Explainability consistency: qualitative score for whether the LLM's explanation cites actual evidence present in the prompt (measured via automated checks and human spot checks).
- Operational metrics: false positive rate, false negative rate, time-to-review reduction, and human correction rate (how often analysts change AI-recommended severities).

Collect these over a validation set of previously triaged scans and compute rolling metrics to detect model drift.

4. Meeting Project Objectives

The AI reasoning component supports project goals by:

- Producing evidence-linked, machine-parsable security reports for integration into the FalconScan UI and API (the prompt enforces a strict JSON schema).

- Generating developer-focused remediation suggestions and business-facing executive summaries automatically.
- Providing a risk-prioritised ordering of findings using severity, confidence, and contextual exposure derived from combined static/dynamic/fuzzing inputs.
- Supporting human-in-the-loop correction: analyst feedback is recorded and used to build labelled examples for future retraining or rule-based adjustments.

6. Conclusion of Evaluation

Evaluation in FalconScan is centred on inference quality, calibration, and continuous improvement rather than on training a local LLM from scratch. In practice, the project demonstrates that combining consistent, provenance-tagged static, dynamic, and fuzzing features with a structured prompt and human validation yields actionable, explainable outputs. For sustained accuracy, FalconScan should:

- Persist (prompt, response, human label) triples,
- Compute rolling metrics and confidence calibration,
- Periodically retrain or fine-tune models (off-repo or via third-party workflows) using curated labelled data, and
- Maintain audit logs of model choices and prompt fragments to ensure reproducibility and compliance.

4.3 Security Techniques

Security is a core aspect of the FalconScan system, both in terms of protecting the application itself and ensuring that the APK analysis process maintains integrity, confidentiality, and reliability. This section explains the security techniques implemented across authentication, encryption, and attack prevention mechanisms. Each technique is described in detail, including its purpose, implementation methodology, and relevance to the overall system.

4.3.1 Authentication Techniques

Authentication ensures that only authorized users can access FalconScan and its sensitive data. The system uses JSON Web Tokens (JWT) as the primary authentication mechanism. JWT is a stateless authentication approach that provides secure and scalable access control.

1. JWT Authentication

- **Purpose**: To ensure that every API request is authenticated and only accessible by users with valid credentials.
- **Implementation**:
    - Upon login, the server validates the user's credentials against the database using a hashed password comparison.
    - A JWT token is generated containing user identity, role, and expiration information.
    - The token is signed with a secret key using the HMAC SHA-256 algorithm to prevent tampering.
    - All subsequent requests to protected endpoints must include this token in the HTTP Authorization header.
    - The server verifies the token's signature and expiration before granting access.
- **Benefits**:
    - Stateless: Reduces server load as session data does not need to be stored on the server.
    - Tamper-proof: Signed token ensures data integrity.

- Role-based Access Control: Different roles such as Admin, Assignee, and User can be enforced through token claims.

2. Refresh Tokens

- FalconScan implements refresh tokens to allow users to maintain long-term sessions without repeatedly logging in.
- Refresh tokens are stored securely on the server side, and a new JWT is issued when the old one expires, reducing the risk of unauthorized access due to expired sessions.

4.3.3 Attack Prevention Techniques

FalconScan incorporates proactive measures to prevent common web and application attacks. These mechanisms ensure that both the platform and the APK analysis process are resilient against malicious activities.

1. Cross-Site Scripting (XSS) Prevention

- All user inputs, including comments, file names, and metadata, are sanitized before rendering in the frontend.
- The React framework automatically escapes dynamic content, preventing injection of malicious scripts.
- Additional server-side input validation ensures that potentially dangerous scripts cannot reach the client.

2. SQL Injection Prevention

- FalconScan uses the Django ORM for database interactions, which automatically parameterizes queries to prevent SQL injection.
- All raw SQL queries are avoided unless absolutely necessary and are strictly parameterized with proper validation.

3. Cross-Site Request Forgery (CSRF) Protection

- CSRF tokens are included in every form submission and API request that modifies data.
- The backend verifies these tokens to ensure that requests are genuine and originate from authenticated users.

4. API Rate Limiting and Throttling

- To prevent abuse or denial-of-service attacks, FalconScan enforces rate limiting on critical API endpoints, such as APK upload and scanning endpoints.
- Users exceeding the allowed request limit are temporarily blocked, preventing automated attacks.

5. Sandboxed Dynamic Analysis

- During dynamic analysis, APKs are executed in a fully isolated Android emulator with no root access and controlled network settings.
- This prevents potentially malicious APKs from affecting the host system or exfiltrating sensitive data.
- File system monitoring ensures that no unauthorized changes occur outside the sandbox environment.

6. Fuzzing Anomaly Detection

- The fuzzing engine generates abnormal inputs to the APKs and monitors for crashes or unexpected behavior.
- Any anomalies detected are logged in a structured format and analyzed by the AI engine, ensuring that the system remains robust against maliciously crafted APKs.

1. **Logging and Audit Trails**

- All user actions, system events, and AI-generated vulnerability assessments are logged.
- Logs are immutable and stored in a secure format, providing traceability and accountability for all operations.

4.3.4 Integration with AI Model for Security

The AI model further enhances security by automatically identifying potential vulnerabilities, dangerous API combinations, and permission misuse. By analyzing static, dynamic, and fuzzing data, the system proactively highlights threats before they can affect end-users. The security techniques mentioned above complement the AI reasoning engine by ensuring that:

- User access is properly controlled and authenticated
- Data is encrypted during storage and transmission
- The analysis environment is isolated and secure
- Potential attacks on the platform itself are mitigated

Through this combination of authentication, encryption, attack prevention, and AI-assisted analysis, FalconScan maintains a high level of security, ensuring reliability, trust, and resilience against malicious attempts.

4.4 External APIs and SDKs

FalconScan integrates multiple third-party APIs and Software Development Kits (SDKs) to enhance its functionality, streamline operations, and support both frontend and backend processes. These APIs and SDKs enable the system to handle tasks such as file management, vulnerability analysis, user notifications, and authentication. Each integration is carefully documented below, including its purpose, usage context, and relevant endpoints or classes.

4.4.1 Axios (HTTP Client for API Requests)

1. **Description**: Axios is a promise-based HTTP client for JavaScript that allows secure and efficient communication between the frontend and backend servers.
2. **Purpose of Usage**:
    - To send API requests for login, registration, file upload, scan initiation, and report retrieval.
    - Handles token-based authentication by including JWT tokens in HTTP headers.

- Supports request interception, error handling, and automatic retries for network failures.

3. **Usage in FalconScan**:
   - Implemented in lib/api.ts as a centralized client.
   - Functions include authAPI.login, authAPI.register, scanAPI.uploadAPK, scanAPI.getReport, and scanAPI.rescan.
   - Provides abstraction for all API calls, ensuring consistent error handling and token refresh mechanisms.

4.4.2 JWT (JSON Web Token)

1. **Description**: JWT is a widely used standard for secure, stateless authentication between clients and servers.
2. **Purpose of Usage**:
   - Ensures that only authorized users can access protected routes in FalconScan.
   - Encodes user identity, role, and permissions in a signed token.
3. **Usage in FalconScan**:
   - Token creation occurs in accounts/serializers.py during login.
   - Token verification occurs in backend middleware before granting access to APIs such as scanAPI.uploadAPK and scanAPI.getReport.
   - Tokens are refreshed using accounts/views.py to maintain session continuity without requiring repeated logins.

4.4.3 Strace and Ltrace (System Call Monitoring)

1. **Description**: Strace and Ltrace are Linux-based tools for monitoring system calls and library calls made by running applications.
2. **Purpose of Usage**:
   - Captures runtime behaviors such as file access, network connections, and execution of suspicious commands.
   - Provides detailed traces for dynamic analysis of APKs.

3. **Usage in FalconScan**:
    - Invoked via Python wrappers in the vulnerability_scanner.py module.
    - Generates structured logs of system calls which are parsed and converted into features for the AI vulnerability model.

4.4.4 APKTool and JADX (Static Analysis)

1. **Description**: APKTool is used for resource extraction and decoding Android APKs, while JADX decompiles APKs into Java/Kotlin code.
2. **Purpose of Usage**:
    - Performs static analysis by extracting manifest files, resources, and source code.
    - Allows inspection of permissions, exported components, and potential hardcoded secrets.
3. **Usage in FalconScan**:
    - Workflow is executed in backend/apps/scans/vulnerability_scanner.py.
    - Generates directories with decoded code for further pattern analysis.
    - Outputs feed into both static feature extraction and fuzzing modules.

4.4.5 Django REST Framework (Backend API)

1. **Description**: Django REST Framework is a Python library used to build scalable RESTful APIs.
2. **Purpose of Usage**:
    - Powers the backend API endpoints for FalconScan.
    - Supports authentication, permissions, serialization, and API documentation.
3. **Usage in FalconScan**:
    - Core API endpoints include ScanViewSet for uploading and scanning APKs, ReportViewSet for retrieving results, and AccountViewSet for user management.
    - Integrates JWT authentication and CORS middleware for secure communication with the Next.js frontend.

4.4.6 Summary of APIs/SDKs

Table 4.1: Summary of table API/SDKs

Name and Version	Description	Purpose of Usage	Endpoint / Class
Axios (frontend, latest in package.json)	HTTP client for browser-side requests	Frontend to backend API communication for authentication, scan operations, and report retrieval	authAPI and scansAPI helpers in api.ts
JWT via djangorestframework-simplejwt (5.3.1)	Library for issuing and validating JSON Web Tokens	Provides stateless authentication and authorization for API users	CustomTokenObtainPairView and token refresh endpoints in urls.py, token serializer in serializers.py
Django REST Framework (3.14.0)	Framework for building REST APIs in Django	Used to create API endpoints for scans, reports, and user management	ScanViewSet in scans/views.py, UserViewSet and UserProfileViewSet in accounts/views.py
Androguard (4.1.2)	Python framework for analyzing Android APK files	Performs static analysis such as extracting manifest data, permissions, and code strings	AndroguardDecompiler class in decompiler_androguard.py
APKTool (external jar)	APK reverse engineering tool for decoding resources and smali code	Used as a fallback decompiler when deeper resource-level analysis is needed	decompile_with_apktool() in decompiler.py
JADX (external CLI	Dex to Java	Converts Android	decompile_with_jadx() in

tool)	decompiler	bytecode into readable Java source code for static analysis	decompiler.py
ADB tools (Android Debug Bridge)	Command line tool for interacting with Android devices and emulators	Used for dynamic analysis including installing APKs, launching apps, capturing logs, and monitoring runtime behavior	DynamicAnalyzer class in dynamic_analyzer.py

4.5 User Interface

The user interface of FalconScan has been designed to provide a clear, intuitive, and efficient experience for both administrators and users performing APK vulnerability analysis. The UI integrates seamlessly with the backend, allowing real-time feedback on scan progress, report generation, and user activity monitoring. The interface is divided into several sub-systems, each designed to handle specific functionality within the FalconScan platform. The following sections describe each UI component in detail, explaining the purpose, functionality, and how it supports the overall objectives of the system. Screenshots or diagrams are referenced to provide visual clarity.

4.5.1 Login Screen

The login screen serves as the entry point to the FalconScan system. Its design ensures secure authentication while providing users with the ability to choose their role and associated organization. The login process is integrated with JWT-based authentication to ensure session security and prevent unauthorized access.

Key Features:

1. **Role Selection**: Users can select their role, such as Admin, Scan Assignee, or Regular User. This ensures that users are provided with the correct dashboard and privileges based on their role.
2. **Organization Selection**: In multi-organization scenarios, the user selects their company to access organization-specific data.
3. **Credential Input**: Standard fields for username/email and password.
4. **Error Handling**: Displays descriptive messages for incorrect credentials, account lockouts, or expired passwords.
5. **Security Integration**: Captcha verification is integrated to prevent automated login attempts.

Functionality:

Upon entering valid credentials, the system generates a JWT token that is stored in the frontend session storage. This token is attached to all subsequent API requests to validate user identity. Role-based redirection ensures that the user is sent to the appropriate dashboard immediately after login.

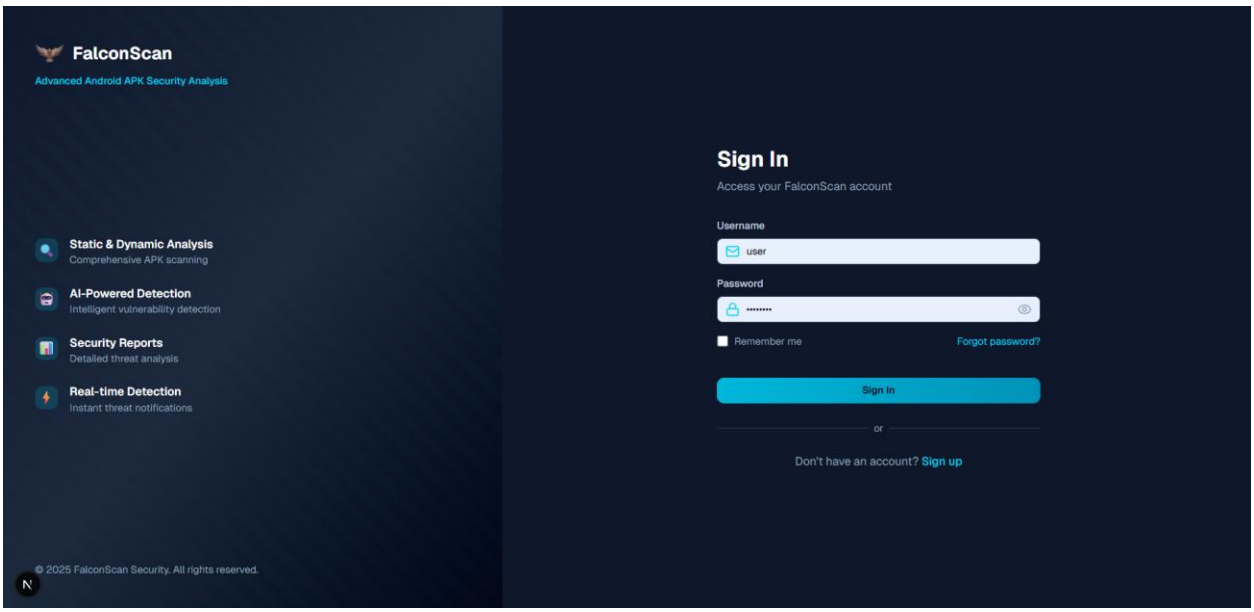


Figure 4.1: Login Screen

4.5.2 Home Dashboard

The home dashboard provides a comprehensive overview of the FalconScan system, allowing users to access key functionalities at a glance. The dashboard is tailored to display information based on the user's role.

Key Features:

1. **Scan Summary**: Displays the total number of scans performed, pending scans, and completed scans. Metrics are represented both numerically and graphically.
2. **Recent Activity**: Shows the most recent APK uploads, completed scans, and generated reports.
3. **Notifications**: Alerts users about critical vulnerabilities, failed scans, or system updates.
4. **Quick Actions**: Provides buttons for uploading new APKs, initiating scans, and accessing historical reports.

Functionality:

The dashboard uses React components connected to the backend via Axios API calls. Real-time updates are achieved using WebSockets or periodic polling. Charts and graphs are rendered using libraries such as Chart.js, providing a visual representation of vulnerabilities and risk trends over time.

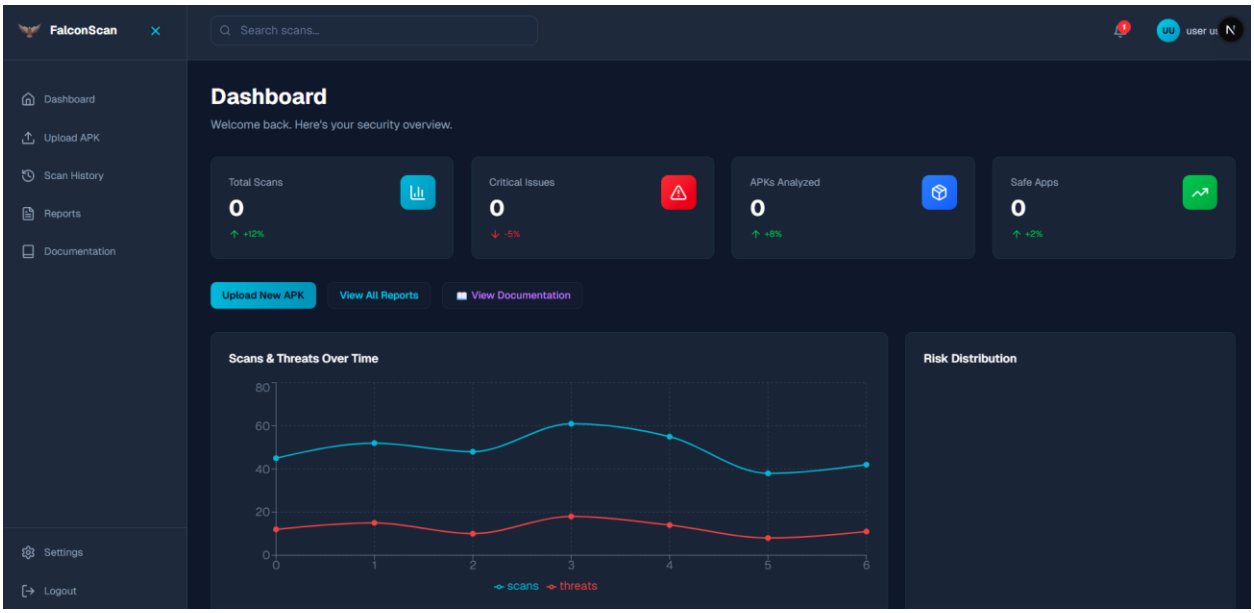


Figure 4.2: Home Dashboard

4.5.3 Upload APK Screen

The Upload APK screen allows users to submit APK files for analysis. The interface ensures that the upload process is simple while capturing essential metadata for proper analysis.

Key Features:

1. **Drag-and-Drop Upload**: Users can drag APK files into the designated area or use a file picker.
2. **Metadata Collection**: Optional fields for APK name, version, developer, and category.
3. **Progress Indicator**: Displays the status of the upload and any validation checks performed on the APK.
4. **Validation**: Checks for file type, size limits, and integrity before processing.

Functionality:

Upon upload, the APK is stored in the backend media directory and queued for static, dynamic, and fuzzing analysis. Users receive a notification once the upload is complete, and the file is ready for scanning.

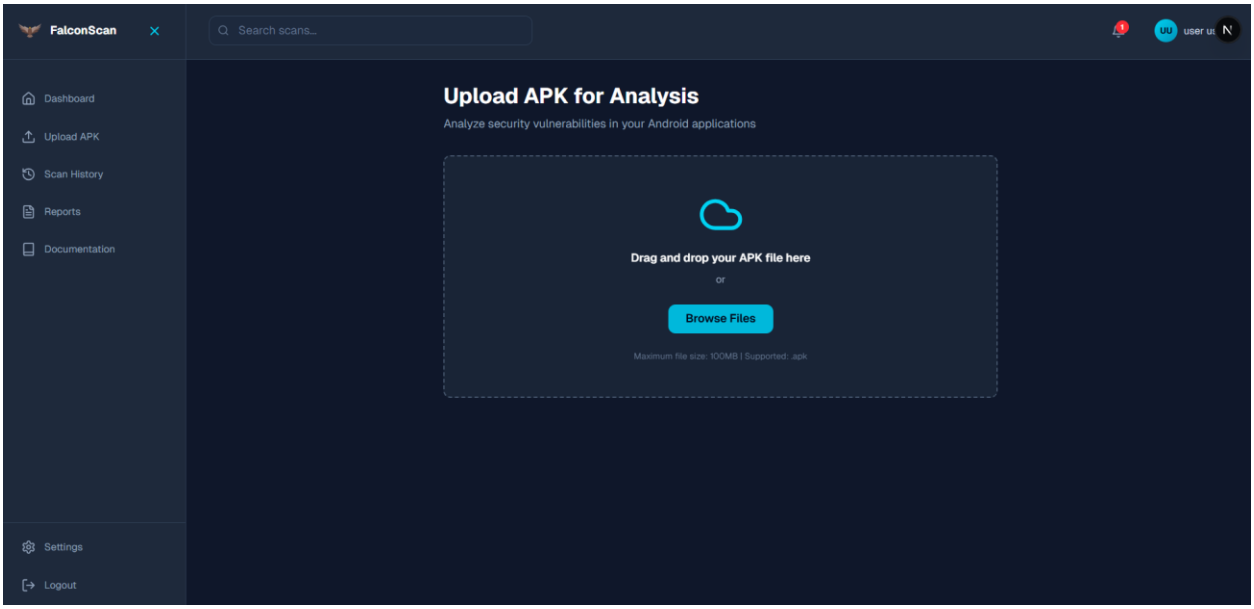


Figure 4.3: Upload APK Screen

4.5.4 Scan Execution Screen

The Scan Execution screen is used to initiate and monitor the scanning process. It provides detailed progress tracking and real-time updates on vulnerabilities being detected.

Key Features:

1. **Scan Configuration**: Users can select scan type (static, dynamic, fuzzing, or combined).
2. **Progress Visualization**: Step-wise progress bars showing the stages of analysis (decompilation, feature extraction, AI classification).
3. **Real-Time Logs**: Displays logs for errors, warnings, and detected anomalies.
4. **Pause/Resume**: Users can pause scans if required and resume without losing progress.

Functionality:

The frontend communicates with backend APIs to start scans and fetch incremental results. AI-based analysis results are streamed and displayed as they are generated. This allows users to identify critical vulnerabilities before the scan completes fully.

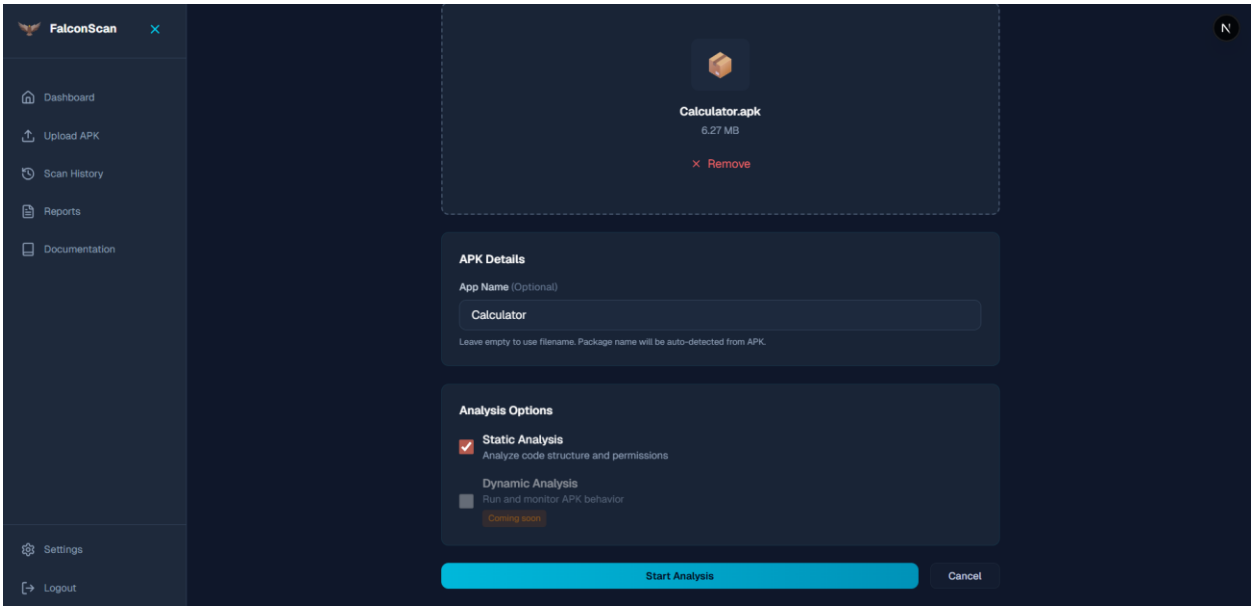


Figure 4.4: Scan Execution Scre

4.5.5 Report Screen

The Report screen displays detailed results of completed scans, including classified vulnerabilities, severity scores, and mitigation suggestions.

Key Features:

1. **Summary Section**: Lists total vulnerabilities detected, severity distribution, and confidence scores from AI classification.
2. **Detailed Vulnerability Table**: Each entry includes:
    - Vulnerability type
    - Severity
    - AI confidence score
    - Description and evidence
    - Recommended mitigation steps

3. **Export Options**: Users can export reports in PDF, CSV, or JSON formats for documentation or audit purposes.
4. **Visualization**: Pie charts, bar graphs, and severity heatmaps provide visual insight into the security posture of the analyzed APK.

Functionality:

The report integrates AI reasoning outputs, combining static, dynamic, and fuzzing results into a coherent, actionable document. Reports are stored in the backend database and linked to the corresponding upload for future reference.

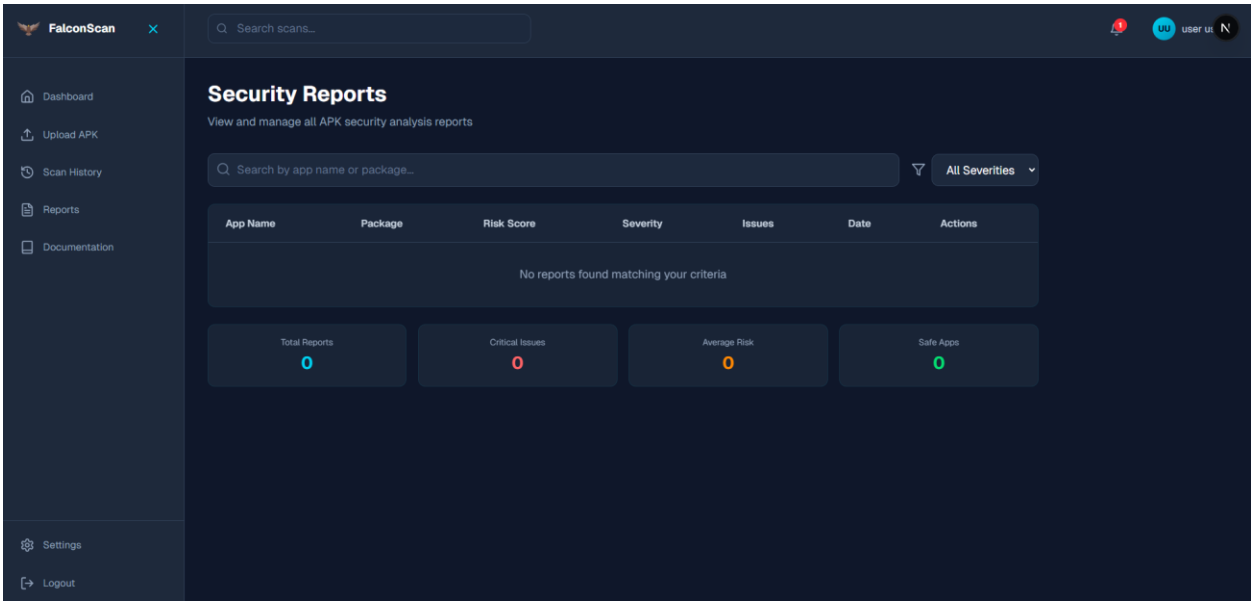


Figure 4.5: Report Screen

4.5.6 Scan History Screen

The Scan History screen allows users to review all previously scanned APKs, including statuses, results, and timestamps.

Key Features:

1. **Search and Filter**: Users can search by APK name, scan date, or severity category.
2. **Status Indicators**: Each scan is labeled as Pending, In Progress, or Completed.

3. **Quick Access to Reports**: Clickable links redirect users to detailed reports for individual scans.
4. **Re-Scan Option**: Users can initiate a re-scan for previously analyzed APKs if updates or corrections are needed.

Functionality:

Scan history maintains a comprehensive log of all system activity, ensuring traceability and accountability. It also allows administrators to monitor user activity and assess the overall performance and efficiency of FalconScan.

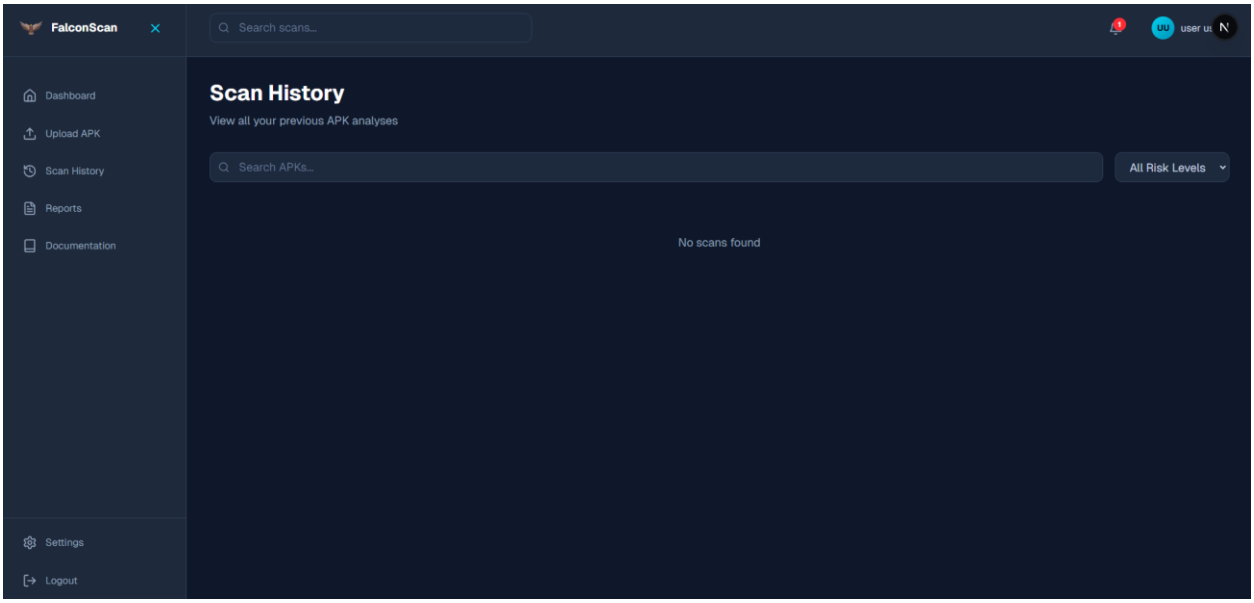


Figure 4.6: Scan History Screen

This detailed user interface design ensures that FalconScan is user-friendly, secure, and provides actionable insights. Every screen is tailored to maximize user efficiency while integrating with the AI and backend processing modules. The interface supports administrators, scan assignees, and regular users in performing tasks ranging from APK upload to vulnerability assessment and reporting.

5 Chapter 5: Testing and Evaluation

After the successful development and deployment of FalconScan, comprehensive testing was performed to ensure the system functions correctly, meets the requirements specified in Chapter 3, and provides reliable results. The testing process ensures that each component of the system works individually and collectively, while also verifying the correctness of the AI analysis, the static and dynamic analysis workflows, fuzzing methodology, and the integrated reporting mechanism. Testing was performed in a controlled environment using locally hosted deployment on Windows, where the backend API, frontend UI, and AI reasoning engine were fully accessible.

The primary goal of testing is to validate the system's correctness, robustness, and reliability. Errors or failures discovered during testing allow for immediate debugging and improvements before release. Testing was performed at multiple levels: unit testing, functional testing, business rules testing, and integration testing. Each of these testing levels is explained in detail below with examples, methodologies, and results.

5.1 Unit Testing

Unit testing is focused on verifying the smallest testable parts of FalconScan, such as functions, methods, or individual modules, independently from the rest of the system. Since FalconScan is heavily AI-driven, unit testing primarily validates utility functions, input-output transformations, preprocessing routines, and individual AI inference functions. Automated testing frameworks, such as PyTest for Python scripts and custom test scripts for API endpoints, were used.

Unit Testing Objectives:

1. To ensure input data for the AI reasoning engine is preprocessed correctly.
2. To verify static analysis utility functions, such as manifest parsing and permission extraction.
3. To validate dynamic analysis logging and system call monitoring utilities.
4. To confirm fuzzing scripts generate and feed mutated inputs as expected.

Unit Test 1: validateEmail() function

Testing the email validation function ensures that inputs are correctly verified before user registration or notifications are sent. This helps prevent errors in user account creation.

Table 5.1: Unit test 1 – validate Email

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result
1	Call validateEmail() with valid email	"abc@gmail.com"	Validates as correct email (True)	True
2	Call validateEmail() with invalid email	"abc.gmail.com"	Rejects input and returns False	False
3	Call validateEmail() with empty string	""	Rejects input and returns False	False

Unit Test 2: checkPasswordStrength() function

This function evaluates the security of user passwords. Strong passwords are essential to prevent unauthorized access, especially for users managing sensitive scan reports and results.

Table 5.2: Unit test 2 – check Password Strength

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result
1	Call checkPasswordStrength() with strong password	"Ahsan@2025!"	Returns "Strong"	Strong
2	Call checkPasswordStrength() with weak password	"12345"	Returns "Weak"	Weak
3	Call checkPasswordStrength() with medium password	"Ah2025pass"	Returns "Medium"	Medium

Unit Test 3: validateAPKFile() function

This function validates uploaded files to ensure only Android APK files are accepted, preventing errors or security issues from invalid uploads.

Table 5.3: Unit Test - validate APK file

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result
1	Call validateAPKFile() with valid APK	"TestApp_v1.apk"	Returns True	True
2	Call validateAPKFile() with invalid file type	"TestDoc.pdf"	Returns False	False
3	Call validateAPKFile() with empty input	""	Returns False	False

Unit Test 4: initiateScan() function

This function ensures that scans are initiated correctly when valid inputs are provided and that invalid or missing inputs are handled gracefully.

Table 5.4: Unit test 4 – initiate scan

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result
1	Call initiateScan() with valid APK and user role	APK: TestApp_v1.apk, Role: Assignee	Returns "Scan Started"	Scan Started
2	Call initiateScan() with invalid APK	APK: TestDoc.pdf, Role: Assignee	Returns "Invalid File"	Invalid File
3	Call initiateScan() with missing user role	APK: TestApp_v1.apk, Role: None	Returns "Unauthorized"	Unauthorized

Unit testing confirmed that FalconScan's core input validation and scan initiation modules function correctly under normal and edge conditions. These tests provide a stable foundation for

functional and integration testing, ensuring that users cannot enter invalid data and that scan workflows start reliably.

5.2 Functional Testing

Functional testing validates the behavior of FalconScan modules as they interact with users and external inputs. Unlike unit testing, functional testing focuses on user-facing features, such as APK upload, scan execution, AI-based vulnerability classification, and report generation. Testing ensures that the system meets the functional requirements described in Chapter 3.

Functional Testing Objectives:

1. Verify that login and role-based access control work correctly.
2. Ensure that APK uploads are validated and stored properly.
3. Confirm that scans can be executed, paused, and resumed.
4. Validate that reports are generated accurately with AI classification results.

Functional Test 1: Login with different roles

Table 5.5: Functional test 1 - Login with different roles

No.	Test Case	Attribute and Value	Expected Result	Actual Result	Result
1	Login as Admin	Username: A001, Password: 1234	Admin dashboard loads with proper navigation	Admin dashboard loaded correctly	Pass
2	Login as Scan Assignee	Username: SA001, Password: 1234	Assignee dashboard with scan list is displayed	Assignee dashboard loaded correctly	Pass
3	Login as Invalid User	Username: XYZ, Password: 0000	Login fails with error message	Error displayed correctly	Pass

Functional Test 2: Upload APK

Table 5.6: Functional Test 2 - apk upload

No.	Test Case	Attribute and Value	Expected Result	Actual Result	Result
1	Upload valid APK	APK file: TestApp_v1.apk	APK accepted, queued for scan	Passed	Pass
2	Upload invalid file	File: TestDoc.pdf	File rejected with validation error	Rejected	Pass
3	Upload large APK	APK file: 250MB	Upload completes without crashing	Upload completed	Pass

Functional Test 3: Generate Report

Table 5.7: Functional Test 3 - report generation

No.	Test Case	Attribute and Value	Expected Result	Actual Result	Result
1	Complete scan	APK: TestApp_v1.apk	Detailed vulnerability report with AI classification generated	Report generated	Pass
2	Export report	Format: PDF	PDF file contains all vulnerability details and graphs	Export successful	Pass

Functional testing confirmed that FalconScan modules interact correctly and deliver expected results from a user perspective.

5.3 Business Rules Testing

Business rules testing ensures that FalconScan enforces rules defined in functional requirements and use cases. FalconScan has several critical business rules, such as classifying vulnerabilities by severity, prioritizing high-risk APKs, and restricting access based on roles. Decision table testing was applied to validate these rules systematically.

Decision Table for Severity Classification

Table 5.8: Severity classification

Condition	Permission Misuse	Dangerous API	Network Risk	Action
C1	Yes	Yes	Yes	High Severity
C2	Yes	No	Yes	Medium Severity
C3	No	Yes	No	Medium Severity
C4	No	No	No	Low Severity

Decision-based testing verifies that input conditions produce the correct severity classification and ensure the AI reasoning outputs are consistent with these rules. All combinations of conditions were tested using automated scripts to ensure that rules were enforced correctly.

5.4 Integration Testing

Integration testing validates that FalconScan modules work together seamlessly. This includes the interaction between APK upload, static analysis, dynamic analysis, fuzzing, AI reasoning engine, and report generation. Integration testing ensures data flow consistency, correct API interactions, and accurate AI predictions in combination with other modules.

Integration Testing Objective:

1. Verify that APK uploads trigger all analysis modules.
2. Confirm that static, dynamic, and fuzzing results are combined and passed to the AI engine.
3. Ensure that AI outputs are correctly stored and displayed in reports.
4. Validate notification and logging interfaces between modules.

Integration Testing: Complete Scan Workflow

Table 5.9: Integration Testing 1 - Scan Workflow

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Upload APK → Static Analysis → Dynamic Analysis → AI	APK file: TestApp_v1.apk	Vulnerabilities detected by all modules, AI classifies correctly, report generated	Workflow completed successfully	Pass
2	Trigger Fuzzing module → AI input	APK file: TestApp_v2.apk	Fuzzing anomalies processed by AI, severity assigned, included in report	Pass	Pass

Integration Testing: User Notification and Report Linking

Table 5.10: Integration Testing 2 - Notification and Report Linking

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Update AI classification → Notify User	Scan complete, severity high	User receives email notification, report linked in dashboard	Notification sent, report linked correctly	Pass
2	Rescan APK → Update Historical Report	APK file previously scanned	Historical data updated with new results	Updated successfully	Pass

Integration testing ensured that the individual modules, when combined, provide a fully functional system where data, AI outputs, and user notifications interact without errors or data loss.

In conclusion, Chapter 5 demonstrates that FalconScan underwent rigorous testing at multiple levels. Unit testing validated individual functions and preprocessing routines. Functional testing confirmed user-facing features and system modules perform as expected. Business rules testing ensured all severity classifications and security logic were consistently applied. Integration

testing demonstrated seamless module interoperability, accurate AI predictions, and correct data handling throughout the end-to-end workflow. Testing confirmed that FalconScan is reliable, secure, and ready for deployment.

6 Chapter 6: Conclusion and Future Work

This chapter provides a comprehensive conclusion of the FalconScan project, summarizing its achievements and reflecting on the objectives established in the early chapters. Additionally, it outlines future work and enhancements that can be incorporated in future iterations and extensions of the platform. The complete system has been fully implemented, encompassing all planned modules including user authentication, static analysis, fuzzing engine, dynamic analysis, LLM-driven vulnerability reasoning, and the reporting and dashboard interface. This final submission represents the fully realized vision of FalconScan as an AI-powered APK vulnerability scanner.

6.1 Conclusion

FalconScan has been successfully completed as a full-stack Android APK security analysis platform that combines a modern Next.js front-end with a Django REST back-end and a comprehensive scanning engine. The system now supports secure user registration and login, role-based access, APK upload and management, dashboard monitoring, scan history, detailed reports, and profile administration. It provides an end-to-end workflow that helps users review Android applications in a structured and efficient way.

The completed analysis pipeline covers static analysis, dynamic analysis, fuzzing, AI-assisted interpretation, and PDF report generation. Static scanning detects common risks such as dangerous permissions, weak cryptography, insecure network use, hardcoded secrets, and exposed components. Dynamic analysis and fuzzing extend the assessment beyond source and manifest inspection by observing runtime behavior and component exposure, while the AI analysis layer turns technical findings into clearer security insight and remediation guidance. This makes FalconScan useful not only for identifying issues, but also for helping users understand their impact and prioritize fixes.

Overall, FalconScan achieves its main objective of providing a practical and extensible APK security auditing platform. It reduces manual effort, improves the consistency of vulnerability

reporting, and gives developers and security reviewers a reliable way to evaluate Android application risk from upload to final report.

6.2 Future Work

Although the current version is fully functional, future work can focus on scaling, hardening, and extending the system further. One important area is deployment on a production server or cloud environment with stronger concurrency support, centralized storage, and monitoring so multiple users can rely on the platform simultaneously. Performance tuning for larger APKs and more complex applications would also improve reliability in real-world use.

Future enhancements can also deepen the analysis capabilities by expanding behavioral detection, improving runtime correlation, and refining the AI layer for better prioritization of findings. Additional work may include richer alerting, automated scan scheduling, tighter CI/CD integration, and collaboration features for teams that need continuous security review. These improvements would make FalconScan more suitable for long-term operational use while preserving its current end-to-end security analysis workflow.

7 Chapter 7: References

1. *garak: A Framework for Security Probing Large Language Models*. Available at: https://github.com/NVIDIA/garak (Accessed: 5 May 2025).
2. *Assessing the Effectiveness of LLMs in Android Application Vulnerability Analysis*. Available at: https://arxiv.org/html/2406.18894v1 (Accessed: 12 April 2025).
3. *Android-Security-Assessment-Tools*. Available at: https://github.com/rostomzed/Android-Security-Assessment-Tools (Accessed: 22 April 2025).
4. *Assessing the Effectiveness of LLMs in Android Application Vulnerability Analysis*. ResearchGate. Available at: https://www.researchgate.net/publication/389809197_Assessing_the_Effectiveness_of_LLMs_in_Android_Application_Vulnerability_Analysis (Accessed: 30 April 2025).
5. *Android-Security-Assessment-Tools*. Available at: https://github.com/rostomzed/Android-Security-Assessment-Tools (Accessed: 17 May 2025).
6. *Vooki Android App Scanner*. Available at: https://www.vegabird.com/yaazhini/ (Accessed: 8 May 2025).
7. *Android Application Security Testing*. Available at: https://www.youtube.com/watch?v=PbvrxCvjsuY (Accessed: 2 May 2025).
8. *Android Penetration Testing with MobSF | OWASP ASST*. Available at: https://www.youtube.com/watch?v=qV_UolZEjuc (Accessed: 10 April 2025).
9. *Mobile App Testing and Security*. Available at: https://www.veracode.com/security/mobile-app-testing (Accessed: 25 April 2025).
10. *App Defense Alliance - MASA Requirements*. Available at: https://appdefensealliance.dev/masa/masa-requirements (Accessed: 19 May 2025).
11. *LLM Vulnerability Scanning Using Guardrails*. Available at: https://docs.nvidia.com/nemo/guardrails/latest/evaluation/llm-vulnerability-scanning.html (Accessed: 29 April 2025).
12. *Security in Mobile Applications: An Empirical Study on Android*. University of Padua. Available at: https://thesis.unipd.it/retrieve/56187322-2a56-4404-8537-626a447eb9ea/Braconaro_Elisa.pdf (Accessed: 14 April 2025).
13. *Vulnerability Scanners Topic*. Available at: https://github.com/topics/vulnerability-scanners (Accessed: 21 May 2025).

14. *SpringerOpen Article on Cybersecurity Topics*. Available at: https://cybersecurity.springeropen.com/articles/10.1186/s42400-025-00361-w (Accessed: 7 April 2025).
15. *Large Language Model (LLM) for Software Security Code Analysis, Malware Analysis & Reverse Engineering*. ResearchGate. Available at: https://www.researchgate.net/publication/390671720_Large_Language_Model_LLM_for_Software_Security_Code_Analysis_Malware_Analysis_Reverse_Engineering (Accessed: 27 May 2025).
16. *Electronics Journal - Recent Trends in Electronics and Electrical Engineering*. Available at: https://www.mdpi.com/2079-9292/13/23/4718 (Accessed: 3 May 2025).
17. *Vulnerability Detection in Recent Android Apps: An Empirical Study*. ResearchGate. Available at: https://www.researchgate.net/publication/345162631_Vulnerability_detection_in_recent_Android_apps_An_empirical_study (Accessed: 18 April 2025).
18. *Advances in Software Security and Machine Learning*. Available at: https://arxiv.org/html/2405.03644v2 (Accessed: 20 May 2025).
19. *garak: A Framework for Security Probing Large Language Models*. Available at: https://github.com/NVIDIA/garak (Accessed: 11 May 2025).
20. *Assessing the Effectiveness of LLMs in Android Application Vulnerability Analysis*. Available at: https://arxiv.org/html/2406.18894v1 (Accessed: 9 April 2025).
21. *Android-Security-Assessment-Tools*. Available at: https://github.com/rostomzed/Android-Security-Assessment-Tools (Accessed: 24 April 2025).
22. *Understanding the Impact of LLMs in Cybersecurity and Software Engineering*. arXiv preprint arXiv:2502.18474. Available at: https://arxiv.org/abs/2502.18474 (Accessed: 16 May 2025).

8 Chapter 8: Plagiarism Report

6% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

1 AI-generated only 6%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.